
Gaze and Stylus Interaction for Precise Object Manipulation in XR: System Design and User Evaluation

Suqi Xiang, 202402600

Xu Han, 202402603

Emmanouil Platakis, 202402638

Master's Thesis, Computer Science

June 2026

Advisor: Ken Pfeuffer



AARHUS
UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE

Abstract

This thesis investigates the trade-off between precision and flexibility in XR input techniques. Mid-air interaction enables flexible 3D manipulation but often suffers from reduced precision, while surface-supported stylus input improves stability and accuracy but is limited by 2D constraints. To address this, we conduct a controlled within-subject study comparing four interaction techniques derived from two design dimensions: selection method (gaze vs stylus) and input space (mid-air vs table-supported interaction).

Participants performed a simplified docking task in XR, and we measured task completion time, selection accuracy, movement characteristics, workload, and fatigue. The results highlight distinct trade-offs between gaze-assisted selection and stylus-based manipulation.

Beyond the controlled study, we implement a VR interior design application that demonstrates how the proposed techniques can be integrated into a room-scale workflow. The system enables users to arrange virtual furniture using seamless gaze-driven transitions between mid-air and surface-supported stylus interaction, without explicit mode switching.

Acknowledgments

Many thanks to Ken Pfeuffer, Yang Liu, and Mathias Lystbæk for offering the opportunity to work on this research project, as well as guiding and supporting us throughout the project. Also, we acknowledge the contributions of all participants of our user study.

*Suqi Xiang, Xu Han, Emmanouil Platakis
Aarhus, June 2026.*

Contents

Abstract	iii
Acknowledgments	v
1 Introduction	1
1.1 Motivation	1
1.2 Methodology	2
1.3 Application Context	3
1.4 Contribution Summary	4
1.5 Use of Generative Artificial Intelligence	5
2 Related Work	7
2.1 XR Interaction Techniques	7
2.2 Surface-supported and Stylus Input	8
2.3 Gaze-based and Multimodal Interaction	8
2.4 Gaze and Stylus Interaction	9
2.5 Adaptive Control-Display Gain in XR	10
2.6 Spatial Proxies and Viewport Control	11
2.7 Interior Design and Spatial Layout in XR	12
3 Concept	17
3.1 Core Idea	17
3.2 Design Problem	17
3.3 Design Space	18
3.4 Input Techniques	18
3.4.1 StylusMidAir	19
3.4.2 GazeMidAir	20
3.4.3 StylusTable	21
3.4.4 GazeTable	22
3.5 Input Space and Output Space	22
3.6 Application-Level Composition	23
3.7 Gaze-Mediated Axis Mapping	23
3.8 Proxies	24

4	Evaluation	27
4.1	Evaluation Goal	27
4.2	Study Design and Conditions	28
4.3	Task Design	28
4.4	Participants	29
4.5	Apparatus and Study Control	30
4.6	Data Logging	30
4.7	Procedure	31
4.8	Measures and Analysis	32
4.9	Results	33
4.9.1	Performance Metrics	34
4.9.2	Movement and Rotation	35
4.9.3	Subjective Ratings	36
4.9.4	Summary	37
5	Application Design	39
5.1	Scenario, Motivation, and Hardware	39
5.2	System Architecture and Interaction State Machine	41
5.3	Direct and Indirect Manipulation	44
5.3.1	Visual-angle gain	46
5.3.2	Re-anchoring on gaze shifts	46
5.3.3	Sticky-gaze buffer	47
5.4	Gaze-Dependent Axis Routing	47
5.5	Proxies, Visual Feedback, and Tools	49
5.6	Implementation Notes and Limitations	51
5.7	Reflection and Lessons Learned	52
5.8	Outlook	53
6	Discussion and Future Work	55
6.1	Overview of Findings	55
6.2	RQ1: Effect of Gaze-Assisted Selection	55
6.3	RQ2: Effect of Surface-Supported Stylus Input	56
6.3.1	Manipulation Precision	56
6.4	RQ3: Trade-offs Between Speed, Precision, Physical Effort, and Visual Effort	57
6.4.1	Speed and Precision	57
6.4.2	Physical Effort and Speed	58
6.4.3	Physical Effort and Visual Effort	58
6.4.4	Subjective Preference and Objective Performance	58
6.5	Summary	59
6.6	Design Implications	59
6.7	Discussion on The Application	62
6.8	Limitations	62
6.9	Future Work	63
7	Conclusion	65

Bibliography	67
A Technical Details of the Application	73
A.1 The Interaction State Machine	73
A.2 Gaze-Dependent Axis Routing	74
A.3 Implicit-Signal Smoothing and Constants	75
A.4 Hardware and Software Stack	75
A.5 Reproduction	76

Chapter 1

Introduction

Extended Reality (XR) systems allow users to interact with digital objects in a three-dimensional space, creating new opportunities for design, training, modeling, education, and spatial productivity [1]. Compared with traditional desktop interfaces, XR enables users to work directly with spatial content and arrange, inspect, and manipulate objects in an immersive environment. However, precise manipulation remains a key challenge in XR [2]. Many tasks, such as positioning, alignment, docking, and layout adjustment, require careful and repeatable control. These requirements are difficult to satisfy with many existing XR input techniques, especially when users need to make small corrections or maintain stable control over time [2–4].

1.1 Motivation

A central tension in XR interaction is the trade-off between spatial flexibility and input precision. Mid-air input, such as hand gestures, controllers, or pen-like devices, allows users to interact directly with spatial content. This makes it suitable for flexible and coarse manipulation, where users can quickly reach, select, or move objects in space. However, mid-air input often becomes unstable during precise tasks. Without physical support, the user’s hand may shake, drift, or become tired during prolonged use. These issues can reduce accuracy and make fine adjustment more difficult. As a result, mid-air input is not always suitable for tasks that require stable, careful, and repeatable manipulation [3].

Surface-supported stylus input offers a different set of strengths. Because the user’s hand can rest on a physical surface, stylus movement can become more stable, accurate, and comfortable [4]. The familiarity of pen input also makes it suitable for controlled movement, fine positioning, and tasks that require repeated adjustment. However, surface-based input is usually designed for two-dimensional movement, while XR interaction often takes place in a spatial environment. This creates a mismatch between the physical input space and the virtual output space [5, 6], exposing a key design challenge: to benefit from the precision and comfort of surface-supported stylus input while still supporting fast target selection and XR manipulation.

Gaze provides a promising opportunity to address this challenge. Users often naturally look at the object or target they intend to manipulate. Therefore, gaze can be used as a fast signal for target selection [7,8], while the stylus can be used for controlled

movement. This separates the roles of eyes and hands: gaze indicates the target, and the stylus performs manipulation. Such a division may reduce the need for precise manual pointing and allow more stable object control. At the same time, gaze-based interaction also introduces challenges, such as tracking inaccuracy, small-target difficulty, and timing issues [9, 10]. Therefore, it is important to evaluate not only whether gaze is useful, but also how it affects performance and user experience when combined with stylus input.

What remains less clear is not whether gaze, stylus input, or surface support can be useful individually, but how these elements interact when combined in the same XR manipulation task. For example, Gaze + Pinch shows how gaze can support target selection in VR, recent pen-and-gaze work explores multimodal object-related interaction in XR, and ComforTable User Interfaces shows that physical surfaces can improve stability, comfort, and exertion in input tasks [4, 8, 11]. However, there is limited empirical evidence comparing gaze-assisted and stylus-only interaction across both mid-air and surface-supported input spaces under the same controlled task. This gap is important because future XR systems may need to combine multiple input modes rather than rely on a single technique.

1.2 Methodology

This thesis investigates gaze and stylus interaction for precise manipulation in XR. The study is guided by three research questions:

RQ1: How does gaze-assisted selection affect acquisition speed, task completion time, and selection errors in stylus-based XR manipulation?

RQ2: How does surface-supported stylus input affect manipulation precision, hand movement, and physical comfort compared with mid-air stylus input?

RQ3: What trade-offs emerge between speed, precision, physical effort, and eye fatigue across gaze-assisted and stylus-only interaction in mid-air and table-based input spaces?

These questions are designed to separate the main factors in the interaction design. RQ1 focuses on whether gaze improves target acquisition compared with stylus-only selection. RQ2 examines whether table support improves manipulation precision and comfort compared with mid-air input. RQ3 considers the broader trade-offs across objective performance and subjective user experience.

To answer these questions, the methodology is based on a controlled comparison of four input techniques derived from two design dimensions: target selection method and physical input space. Target selection is performed either by stylus pointing or by gaze, while manipulation is performed either in mid-air or on a physical table surface. Crossing these two dimensions produces four techniques: *StylusMidAir*, *GazeMidAir*, *StylusTable*, and *GazeTable*.

This comparison is used because the thesis aims to understand how gaze assistance and surface support affect precise XR manipulation. Gaze may reduce the need for precise manual pointing, while surface support may improve stability during manipulation. However, these two factors may also introduce costs, such as gaze timing issues, eye fatigue, slower movement, or input-output mapping problems. Comparing the four techniques under the same task makes it possible to separate the effect of gaze-assisted

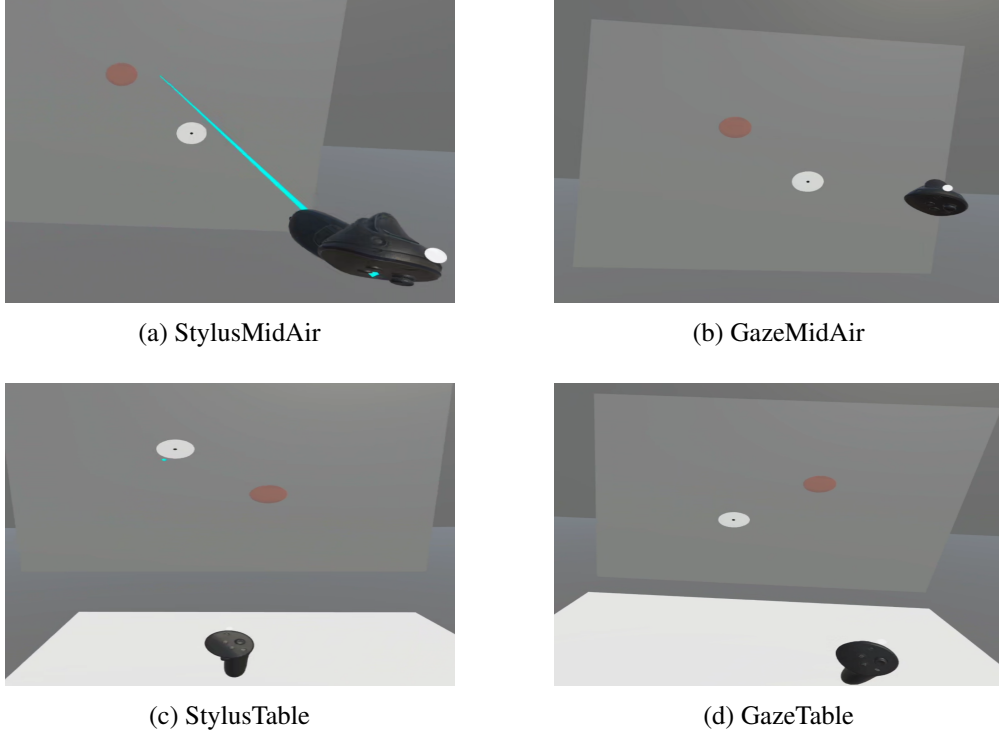


Figure 1.1: Figures Showing the Four Techniques When Idle

selection from the effect of table-based input.

We conducted a controlled within-subject user study using a simplified plane-based docking task. The task was simplified from full 3D object manipulation in order to isolate fundamental input performance rather than evaluating a complete application workflow [12, 13]. In each trial, participants moved an interactable object to a target position under different technique, distance, and target-size conditions. This task allowed the study to measure acquisition time, manipulation time, task completion time, positional error, failed selections, stylus movement, stylus rotation, workload, eye fatigue, and user preference under comparable conditions.

1.3 Application Context

Beyond the controlled comparison of input techniques, the thesis also situates these interactions within an application design context. This application component extends the controlled study into a more realistic workflow and demonstrates how the two design dimensions those techniques span - selection method and input space - can be instantiated in a room-scale XR scenario, with one application-level manipulation mode - gaze plus pen-on-desk with gaze-driven axis routing - that the controlled study did not directly measure.

The concrete instantiation that the thesis evaluates is a hybrid VR interior-design application, in which the user spawns and arranges virtual furniture inside a 2 m x 2 m room while seated at an ordinary desk. The same physical desk supports the user’s hand, and the same stylus tip serves both as a mid-air 6-DOF tool and as a desk-supported

precision tool, with implicit transitions between the two governed by where the user is looking. The user never opens a mode menu, never holds a modifier button, and never needs to recall what state the system is currently in.

Building such an application surfaces a problem that the controlled docking study cannot expose on its own: the dimensional gap between a 2D pen on a flat desk and a 3D scene with at least three degrees of freedom. When the user pushes the stylus forward across the desk, the system has to decide whether the corresponding object motion is up (along the world Y axis) or away (along Z). Existing surface-pen designs answer this by fixing the mapping ahead of time [14], by adding a second tracked surface [15], or by asking the user to flip an explicit toggle [16]. The application takes a different position: the user’s gaze - both its current target and the vertical angle of the gaze ray - together with whether the pen is pressed onto the desk, are sufficient to route the missing axis in real time, without an explicit mode switch and without additional hardware.

Two further mechanisms make this routing tractable in a room-scale workspace. A 2D shadow proxy projects a flat handle for every interactable object onto the desk, turning the desk into an addressable surface for objects that float well outside arm’s reach. An on-pen minimap proxy, anchored to the controller and activated by looking down at the pen, provides a forward perspective for depth refinement without forcing the user to physically walk around the object. Together, the two proxies extend the design space characterized in the study into a continuous workflow that scales to a full virtual room operated from a single seated position.

Interior design is a deliberate choice of scenario: it is the canonical XR object-manipulation task, immediately recognizable to non-expert users, and it stresses both coarse placement, such as dropping a sofa into the room, and fine alignment, such as sliding it flush with a wall, within the same session.

1.4 Contribution Summary

In summary, this thesis makes the following contributions:

- A comparative design space for gaze-assisted and stylus-only interaction across mid-air and table-based input spaces.
- An implementation of four XR manipulation techniques: **StylusMidAir**, **GazeMidAir**, **StylusTable**, and **GazeTable**.
- A controlled user study that compares the four techniques and evaluates how gaze selection and surface support affect speed, precision, hand movement, comfort, eye fatigue, and user preference.
- Design insights for combining gaze, mid-air stylus input, and surface-supported stylus input in future XR systems.
- An application design context that demonstrates how the design space of the four input techniques can be instantiated as a hybrid VR interior-design workflow.

1.5 Use of Generative Artificial Intelligence

For the written part of this thesis, Anthropic’s Claude was used to a limited extent as a writing and development assistant. The model was used for editing and polishing prose into a more formal academic register, for occasional suggestions on structure and phrasing, and for generating preliminary SVG sources of the UML class diagram, the finite-state-machine diagram, and the axis-routing diagram of Chapter 5. During the implementation of the application, the model was also used for small optimizations and refactoring of code the author had already written, debugging help on specific runtime errors, look-ups of Unity API usage and idiomatic C# patterns, short utility-function boilerplate, and code comments. For the implementation of the prototype of techniques and the tasks in Unity, Claude was used to help debug the errors in the code.

All AI-assisted text was reviewed, edited, and verified by the author before inclusion, and all AI-assisted code changes were tested in context before being committed. The authors takes full responsibility for the final content of the sections listed above.

Chapter 2

Related Work

2.1 XR Interaction Techniques

Prior work in XR interaction has explored a wide range of techniques for selecting and manipulating virtual objects, including hand gestures, handheld controllers, ray pointing, and pen-like devices [2, 17]. These techniques are widely used because they preserve one of the central strengths of XR: users can interact with virtual content directly in three-dimensional space instead of through a fixed desktop interface. For coarse object manipulation, this direct spatial control is effective. Users can reach toward objects, point at them, or move controllers through space in a way that roughly matches the intended virtual movement.

However, direct mid-air interaction also introduces precision problems. Since users do not have physical hand support, small hand tremors, controller drift, and fatigue can affect the final position of the manipulated object. These issues become more serious in tasks that require repeated fine correction, such as alignment, docking, layout adjustment, or precise placement [3]. Ray-based techniques can reduce the need to physically reach distant targets, but they introduce another form of instability: small angular movements of the controller can produce large object movements at distance. In addition, pressing a controller trigger may slightly disturb the controller pose at the moment of selection or release, which is problematic for precise manipulation.

Recent XR interaction techniques try to reduce these problems in different ways. For example, work on handover-style interaction explores how users can transfer objects between interaction states or input modes, supporting flexible manipulation across different spatial contexts [2]. RestfulRaycast addresses the fatigue problem by making ray-based interaction more physically comfortable, reducing the need for continuous unsupported arm movement [17]. These systems show that mid-air input can be improved, but they still operate mainly within an unsupported spatial input paradigm. They do not fully solve the problem of how to combine mid-air flexibility with the physical stability needed for precise object placement.

This thesis takes the same focus on XR object selection and manipulation as a basic interaction problem, and similarly treats spatial flexibility as an important requirement: users should be able to reach, select, and move virtual objects without being restricted to a traditional desktop interface. However, rather than assuming that mid-air input should remain the primary input space for both coarse and precise manipulation, we compare

mid-air stylus input with surface-supported stylus input under the same controlled task. This makes it possible to examine whether the flexibility of mid-air interaction can be balanced with the stability of table-based input.

2.2 Surface-supported and Stylus Input

Surface-supported input provides one alternative to unsupported mid-air interaction. Physical surfaces can provide tactile support, reduce hand tremor, and improve input stability, which makes them useful for controlled and precise movement [4, 18]. Stylus input is especially relevant because it is already associated with precision-oriented activities such as writing, drawing, sketching, annotation, and design. When used on a surface, the stylus benefits from both the familiarity of pen input and the physical support of the table.

The ComforTable User Interfaces work is particularly relevant because it shows that physical surfaces can reduce exertion and improve comfort in mid-air or near-surface input tasks [4]. Its main insight is that the body does not interact with virtual objects in isolation: the physical environment around the user can be used as part of the input system. Similarly, studies of stylus and touch input show that surface-based input can provide high precision because the hand and wrist are physically stabilised [18]. These findings support the use of a table as a stabilising substrate in our table-based techniques.

However, surface-supported input also creates a central mapping problem. A stylus moving on a table naturally produces two-dimensional movement, while XR content often exists in a three-dimensional space. The system must therefore decide how movement on the physical surface corresponds to movement in the virtual world [5]. Prior work on physical pads and indirect input planes, such as ReachPad and ARPads, shows that surface-based mappings can extend reach and support indirect manipulation, but they also require careful design so that users understand how their physical movement affects virtual object movement [6, 19]. If the mapping is too fixed, it may not adapt well to different object positions or task contexts. If it is too indirect, users may lose the feeling of direct control.

This thesis adopts the view that physical surfaces can support more stable and precise input than unsupported mid-air movement, and our table-based techniques similarly use the physical table as a stabilising input surface for stylus movement. We do not, however, treat surface input as a standalone solution. Surface support improves physical stability but also introduces the problem of how two-dimensional stylus movement should be interpreted in a spatial XR environment. We therefore evaluate surface support as one dimension in a broader design space that also includes gaze-assisted selection, examining not only whether surface-supported stylus input is precise but also how it performs when combined with gaze-based target selection and compared directly against mid-air stylus baselines.

2.3 Gaze-based and Multimodal Interaction

Gaze interaction has been widely studied in XR because gaze often reflects the visual attention and intended target of the user [7]. In many interaction tasks, users look

at an object before selecting or manipulating it. This makes gaze a useful signal for target acquisition. Gaze-based and gaze-assisted techniques use the eyes to indicate a candidate target, reducing the need for precise manual pointing [8, 20]. For example, Gaze+Pinch separates target indication and confirmation: gaze identifies the likely target, while a pinch gesture confirms the selection [8]. This division reduces manual aiming effort because the hand no longer needs to point precisely at the target.

This eye-hand division of labor is important for XR because gaze and hand input have different strengths. Gaze is fast and can move between distant targets with little physical effort, but it is not stable enough for continuous precise manipulation. Hand input is slower for target acquisition, but provides more deliberate control once the object has been selected. For this reason, gaze is often more useful as an intent signal than as a standalone manipulation method. It can tell the system what the user is likely interested in, while another input channel performs the actual confirmation or manipulation.

At the same time, gaze interaction has important limitations. Eye tracking can be affected by calibration error, tracking noise, and limited precision, especially for small or closely spaced targets. Timing is also a problem: gaze may move away before the confirmation action is completed, producing late-trigger or wrong-target errors [9, 10]. These limitations explain why gaze is usually more robust in multimodal interaction than as a single input channel. In multimodal interaction, gaze provides a fast but probabilistic signal of intention, while the hand or stylus provides deliberate control.

This thesis takes the same approach of treating gaze as an intent signal rather than as a complete manipulation method. In our techniques, gaze does not move the object directly; instead, it indicates which object the user intends to manipulate, while the stylus controls the movement. Beyond this division of labor, we evaluate the role of gaze across two different physical input spaces. Prior gaze-based work shows that gaze can support fast selection but does not fully explain whether this benefit changes when the hand is unsupported in mid-air versus physically supported on a table. We therefore examine gaze assistance not only as a selection technique, but also as a factor whose effect may depend on the surrounding input space..

2.4 Gaze and Stylus Interaction

Multimodal gaze and stylus interaction is particularly relevant to this thesis because it combines fast visual targeting with precise manual control. Previous work has shown that gaze can modulate pen and touch interaction, allowing users to switch between direct and indirect input based on visual attention [21]. In this type of interaction, gaze is not simply another pointing device. Instead, it changes how the system interprets the pen input. For example, when the user looks at the pen or touch surface, the system can treat the input as direct surface input; when the user looks at a distant object or display area, the same pen movement can be interpreted as indirect input. This shows that gaze can function as a routing signal between different input reference frames.

More recent XR work has investigated pen-and-gaze interaction techniques for object-related manipulation tasks [11]. These systems show that gaze and stylus input can take complementary roles: gaze can identify the object or region of interest, while the stylus provides a controlled manual channel for manipulation. This is especially

relevant for XR because object manipulation often requires both fast target acquisition and precise adjustment. A user may first need to quickly indicate which object should be manipulated, and then make small corrections to its position or orientation. Combining gaze and stylus input can support this workflow by separating visual intention from manual control.

However, existing gaze-and-stylus work does not fully isolate the combined effects of target selection method and physical input space. In particular, it remains unclear how gaze-assisted and stylus-only interaction compare when both mid-air and table-based input are evaluated under the same controlled task. It is also unclear whether the benefit of gaze comes mainly from faster target selection, reduced manual aiming, or the way gaze changes the interpretation of stylus input. This matters because gaze may not only select a target; it may also help the system decide which object, reference frame, or movement axis the stylus input should control.

A more recent line of work has begun to use the stylus itself as the mode-switching signal in pen-plus-gaze systems. Liu et al.’s StylusPort [22] encodes the transition between manipulation and locomotion as a physical flip of the stylus end-over-end, with gaze providing the destination of the resulting teleport. The architectural commitments are the same as those of the application of Chapter 5 — a single stylus across all modes, mode transitions encoded by physical signals on the device rather than by menus or modifier buttons, and gaze as the always-on routing signal — but the target of routing differs: StylusPort routes the user’s position, whereas our application routes the world axis onto which the 2D pen delta is projected. A unified system that handles both manipulation and locomotion through the same architectural pattern is treated as future work rather than as a current contribution.

This thesis takes the same basic division of labor between eye and hand: gaze supports target indication, while stylus input supports controlled manipulation. However, rather than evaluating one combined gaze-and-stylus technique in isolation, we compared four techniques in a 2×2 design space: gaze-assisted versus stylus-only selection and mid-air versus surface-supported input. This structure allows the study to examine not only whether gaze and stylus can be combined, but also when the combination is useful and what costs it may introduce. It also prepares the ground for the application-level routing problem developed later: if gaze can indicate not only what the user wants to select but also how the system should interpret stylus input, then gaze can become a routing signal for hybrid XR manipulation rather than only a target-selection signal.

2.5 Adaptive Control-Display Gain in XR

In an XR workflow that operates across more than one working distance, a single fixed control-display (CD) gain becomes the wrong choice. A gain calibrated for arm’s-reach manipulation overshoots when the user reaches at distance; a gain calibrated for distant reach produces sluggish micro-adjustments up close. The trade-off has been examined for several decades. Casiez et al. ran a large empirical study of mouse, touch, and stylus pointing tasks in 2008 and reported that adaptive CD gain consistently outperforms any single fixed value across distance ranges, with no measurable workload cost [23]. Their finding has motivated a wave of XR-specific adaptive-gain work, which can largely be

sorted by what signal the adaptation actually reads.

Liu et al. read the manipulated object’s absolute distance from the user [24]. Their framework partitions reach space into discrete distance ranges, applies a calibrated translation-and-rotation gain coefficient per range, and transitions smoothly between ranges as the manipulated object moves. The model targets both spatial overshoot at close range and arm fatigue at distance, and their controlled study reports measurable improvements in completion time and accuracy compared to a fixed-gain baseline. The cost is a per-range calibration step that the system has to maintain.

PinchLens reads a different signal entirely [25]. The technique targets the failure mode of fixed gain in dense small-target scenes, combining a localised spatial magnification with an adaptive CD gain that decelerates the virtual hand relative to the physical hand as the user approaches a magnified region. The adaptation is triggered by entering a target-rich cluster rather than by an absolute distance reading, and the evaluation shows meaningful improvements on dense small-target selection. The contribution is that adaptive gain can be invoked contextually - by proximity to a cluster the system knows is important - rather than continuously.

Our application’s gain (§5.3.1) reads a third signal: the ratio of two distances, eye-to-object over eye-to-pen, computed every frame from viewing geometry. This formulation has the side effect of requiring no calibration step at all - neither distance ranges to tune nor a magnified region to detect - because the angular relationship between the pen’s motion and the object’s motion as seen from the user’s eye is structurally invariant to where each of the three points lies in space. It is not a magnification technique; the interior-design workflow demands a continuous distance range from arm’s reach to the far wall of a $2\text{ m} \times 2\text{ m}$ room, and the discrete cluster of small targets that PinchLens optimises for is not the shape of the problem we face. The lineage from Casiez et al. to Liu et al. to PinchLens establishes that fixed gain is the wrong default; our contribution is the version of that principle in which the gain function is determined entirely by the user’s own viewing geometry and needs no per-task tuning.

2.6 Spatial Proxies and Viewport Control

Seated XR workflows have to contend with content that lies beyond the user’s immediate reach. Walking to it conflicts with the seated posture; ray-pointing sacrifices precision; instantiating a copy nearby creates state-management overhead. The family of designs that has emerged in response — proxies that project virtual content into a reachable workspace — spans more than three decades and frames much of what our application’s shadow and minimap proxies (§5.5) have to take a position on.

The earliest position in the family is Herndon et al.’s interactive shadows [26]. In their system, each 3D object’s 2D shadow on a reference plane doubles as an interactive widget: grabbing the floor shadow constrains the manipulation to slide the object along the X-Z plane, grabbing the wall shadow constrains it to X-Y. The shadow is simultaneously a depth cue (the gap between object and shadow encodes height) and an axis-constrained manipulation handle, and Herndon et al.’s argument is that the same projection the user reads for spatial information can be re-purposed as a manipulation surface at no additional UI cost. The widget in their formulation is invoked — the user grabs the shadow for a specific manipulation step, after which the shadow

returns to being a passive depth cue. The shadow proxy of our application makes the opposite temporal choice: it is permanently active for every Interactable, visually indistinguishable from a naturalistic ground shadow, and therefore costs nothing to the user’s attention until the pen actually engages it. World-in-Miniature by Stoakley, Conway, and Pausch’s, introduced three years later [27], took the proxy idea to the other extreme. Their hand-held scaled-down duplicate of the entire virtual environment is permanent and foreground, continuously displayed, continuously available, occupying the non-dominant hand and a substantial fraction of the visual workspace whether the user is currently manipulating something or not. WiM’s longevity in the literature comes from the strength of its proprioceptive coupling: the user’s spatial reasoning about the scene is anchored to the same hand the proxy is anchored to. The cost is the continuous attentional load, and a large body of subsequent work - slice-based volumetric variants, transitional AR layout systems, tablet-mediated extensions – has been about reducing that cost while keeping the proprioception argument intact.

Recent body-anchored viewports continue that line. Wang et al.’s Handows [28] attaches interactive multi-window interfaces to the user’s non-dominant palm, with thumb gestures summoning, switching, and dismissing windows on demand. The palm anchors the proxy proprioceptively and serves as a coarse haptic substrate for input. Their evaluation reports significantly reduced physical effort and head movement compared to floating-window baselines, and faster window-switching for precise alignment tasks. The proxy is no longer always-on, but the non-dominant hand is still occupied for the duration of every interaction it hosts.

Our application uses two proxies simultaneously, and they sit at different points in the design space these systems span. The shadow proxy inherits Herndon’s conceit that a 2D handle on a flat plane is a useful constrained-manipulation surface, but moves it from invoked widget to ambient feature — one per Interactable, spawned at scene start, always visible, costing nothing until engaged. The on-pen minimap is closer to a Handows-style body-anchored viewport, but anchored to the dominant hand (the same hand holding the pen) and gated by gaze rather than by gesture: it appears when the user’s gaze settles on the pen during indirect manipulation and dismisses automatically when the gaze leaves. The combination of the two — permanent-and-ambient plus transient-and-foreground — covers two distinct cost profiles within the same workflow, because the proxies activate under disjoint conditions and never compete for the user’s attention.

A nearby family pairs the headset with a held tablet rather than with a body-anchored viewport. SymbiosisSketch [15] routes detailed 3D sketching onto a tracked tablet’s 2D surface in situ; VRSketchIn [29] formalises a design space of pen-and-tablet interaction in VR through user-drawn portals that project the tablet plane into the scene. Both share the proxy-plus-precision-surface intuition with WiM and Handows, but commit the non-dominant hand to a held device for the duration of every fine interaction. Our application declines that binding and uses the ordinary desk the user is already at.

2.7 Interior Design and Spatial Layout in XR

Interior design and architectural layout are canonical XR application domains: they require spawning, positioning, rotating, and aligning a moderate number of virtual

objects in a room-sized scene, and they stress both coarse 3D placement (dropping a couch into a corner) and fine sub-centimetre alignment (sliding it flush against a wall) within the same session. Commercial XR systems have built around this use case but tend to commit to a single dominant input paradigm: IKEA Place to consumer AR furniture preview, Gravity Sketch to free-form mid-air design, Shapr3D to tablet-based precision CAD. The research literature has explored a wider design space, organised loosely around three recurring strategies — hybrid devices, dynamic physical haptics, and implicit gaze- or context-driven routing — and our application’s choices are positioned against representative work from each.

The hybrid-device strategy pairs the headset with a second tracked surface that hosts the precision input. TouchWIM [30] is the most recent representative: an AR headset paired with a tablet that displays a scaled miniature of the room, with users arranging the layout by touching the miniature. The reported study finds TouchWIM significantly more efficient than HMD-only baselines such as hand-ray plus direct touch, which is consistent with the surface-precision result we develop empirically in Chapter 4. The cost is that the non-dominant hand is bound to the tablet for every fine interaction, and the user has to coordinate a miniature view with the room-scale scene. SymbiosisSketch [15] and VRSketchIn [29], discussed in §2.6, sit in the same hybrid-device family with held tablets as the precision substrate.

The dynamic-haptics strategy goes further still. RoomShift [31] uses a swarm of furniture-moving robots to physically reconfigure the user’s environment so that real-world layout matches the virtual scene at all times. The result is whole-body haptic feedback during architectural review: users can sit on virtual chairs, lean against virtual walls, and the system rearranges the physical scene to maintain the illusion. The hardware footprint — nine instrumented robots and a substantial floor area — places RoomShift outside the consumer deployment envelope, but it demonstrates the extreme that prior work has been willing to take in the search for haptic grounding at room scale.

A separate line of work shifts the burden from physical hardware to perceptual or semantic signals. EyeGuide and EyeConGuide [32] use eye gaze to dynamically spawn visual alignment grids near the user’s intended sketch location, reducing the cognitive load of placing constraints manually during 3D sketching, and demonstrating that anticipatory gaze can reveal interaction intent in time to be useful for downstream guidance. Gaze&Wall [33] takes a complementary position: the intersection of the user’s gaze ray with a hand-controlled plane resolves the depth ambiguity of ray-cast selection in densely populated scenes. The semantic strand, exemplified by Wang et al.’s scene-context-aware system [34], exploits object semantics and inter-object relationships (a chair’s typical proximity to a desk, a lamp’s affinity for a table surface) to predict user intent and propose context-appropriate placements, evaluated through a furniture-arrangement application that compares controller, head-pose, and gaze input.

Our application draws on each of these strategies without committing to any of them in its pure form. It grounds the interaction with a physical surface, as TouchWIM and RoomShift do, but the surface is the user’s ordinary desk rather than a held tablet or a fleet of robots, and the non-dominant hand is freed for a wrist-anchored tool menu rather than bound to a precision device. It treats gaze as an implicit signal that reveals interaction intent, as EyeGuide and Gaze&Wall do, but uses gaze to choose the active manipulation mode and to route the 2D pen delta onto a 3D world axis rather than to spawn visual scaffolding or to disambiguate ray selections. It operates in the interior-

design domain that Wang et al. studied, but does not rely on scene semantics to predict placements; the routing decisions are made entirely from signals the user produces during the task. The combination of surface-supported precision input, implicit gaze-driven axis routing, and a layered shadow-plus-minimap proxy strategy in a single seated workflow on commodity hardware — without an explicit mode UI, a second held device, or scene-specific priors — does not appear in any of the surveyed systems.

Table 2.1: Shared foundations and contributions of this thesis.

Topic	What we share	What we depart from
XR interaction techniques (§ 2.1)	Focus on XR object selection and manipulation; spatial flexibility as a core requirement	Does not assume mid-air input is the sole primary space; directly compares mid-air and surface-supported stylus input under the same controlled task
Surface-supported & stylus input (§ 2.2)	Physical surfaces improve input stability and precision; ordinary desk used as a stabilising substrate for stylus movement	Surface input is not evaluated in isolation; it is one dimension in a broader 2×2 design space combined with gaze-assisted selection
Gaze-based & multimodal interaction (§ 2.3)	Gaze used as an intent signal rather than a full manipulation method; eyes indicate the target while the hand controls movement	Evaluates gaze assistance across two different physical input spaces to determine whether the gaze benefit changes with the input space
Gaze and stylus interaction (§ 2.4)	Division of labour between eye and hand: gaze for target selection, stylus for controlled manipulation	Systematic 2×2 comparison that explicitly asks when the combination is beneficial and what costs it introduces
Adaptive CD gain in XR (§ 2.5)	Rejection of fixed gain; per-frame adaptive rule; no per-task manual tuning required	Signal driving adaptation differs: visual-angle gain (eye-to-object / eye-to-hand) replaces absolute distance or target proximity, making the rule scale-invariant across a continuous near-to-far range
Spatial proxies & viewport control (§ 2.6)	2-D ground-plane projection as manipulation handle; body-anchored viewport; single seated-position use; rejection of held tablets	Combines two proxy regimes simultaneously: a permanent ambient shadow proxy and a gaze-triggered minimap proxy, active under disjoint conditions at no cognitive cost
Interior design & spatial layout in XR (§ 2.8)	Physical surface improves XR manipulation precision; gaze as an implicit intent signal that drives routing without explicit gestures	No held tablet, no robotic hardware, no scene-semantic priors; repurposes an ordinary uninstrumented desk as the haptic substrate; combines surface precision input, implicit gaze routing, and layered proxy strategy in a seated workflow on commodity hardware

Chapter 3

Concept

This chapter presents the conceptual design of the thesis. The first part of the chapter focuses on the four input techniques evaluated in the controlled user study: *StylusMidAir*, *GazeMidAir*, *StylusTable*, and *GazeTable*. These techniques are organized around two design dimensions: target selection method and physical input space. The later sections describe how these techniques are composed in the application-level system design.

3.1 Core Idea

The core concept of this thesis is to investigate gaze-assisted stylus interaction for precise object manipulation in XR [8, 11]. The concept combines three interaction resources: gaze, stylus input, and physical surface support. Gaze is used as a fast signal for indicating the user's intended target, the stylus is used as a controlled input device for manipulation, and the physical table provides support for stable hand movement.

Instead of treating these input resources separately, this thesis combines them into a comparative design space. This design space results in four input techniques: *StylusMidAir*, *GazeMidAir*, *StylusTable*, and *GazeTable*. The two dimensions of this space are described in detail in Section 3.3.

The purpose of this concept is not to propose a single best technique, but to compare how different combinations of gaze, stylus input, mid-air movement, and table support affect XR manipulation. The four techniques allow the study to examine the role of gaze-assisted selection, the effect of surface-supported manipulation, and the trade-offs between speed, precision, comfort, and control.

3.2 Design Problem

The design problem addressed in this thesis is how to support precise manipulation in XR without losing the flexibility of spatial interaction. Precise manipulation requires stable and repeatable control, but XR interaction often takes place in open three-dimensional space, where the user's hand lacks physical support [3,4]. This makes small corrections difficult and can increase physical effort during prolonged manipulation.

Surface-supported stylus input can improve stability by allowing the hand to rest on a table [4, 18]. However, it also introduces an input-output mapping problem. The user's movement occurs on a two-dimensional physical surface, while the manipulated

object is displayed in a virtual spatial environment. The system must therefore define how movement on the table should correspond to movement of the virtual object.

Gaze-assisted selection introduces another challenge. Gaze can reduce the need for precise manual pointing, because users often look at the object they intend to manipulate. However, gaze input may be affected by tracking inaccuracy, timing issues, and visual fatigue [9, 10]. Therefore, gaze may improve target acquisition in some cases, but it may also introduce new costs.

The design problem is therefore not simply how to add gaze or table support to stylus interaction. Instead, the problem is how to understand the interaction between these factors: whether gaze improves selection, whether surface support improves precision, and how these benefits are balanced against possible costs in speed, effort, and user comfort.

3.3 Design Space

The four techniques are organized as a 2×2 design space [4, 11]. The first dimension is the target selection method: selection can be performed either by stylus pointing or by gaze. The second dimension is the physical input space: manipulation can be performed either in mid-air or on a physical table surface.

	Mid-air input	Table-based input
Stylus selection	<i>StylusMidAir</i>	<i>StylusTable</i>
Gaze selection	<i>GazeMidAir</i>	<i>GazeTable</i>

Table 3.1: A 2×2 design space formed by target selection method and physical input space.

This design space allows the study to isolate the effects of gaze selection and surface support. Comparing *StylusMidAir* with *GazeMidAir* shows how gaze-assisted selection affects mid-air manipulation. Comparing *StylusTable* with *GazeTable* shows how gaze-assisted selection affects table-based manipulation. Comparing the mid-air techniques with the table-based techniques shows how surface support affects speed, precision, hand movement, and comfort.

3.4 Input Techniques

The four input techniques in this section are evaluated independently in the controlled user study. Each technique is a self-contained way of selecting and manipulating an object.

In the following descriptions, the term stylus refers to the input device used for pointing and surface-based manipulation. The implementation details of this device are described in 4.

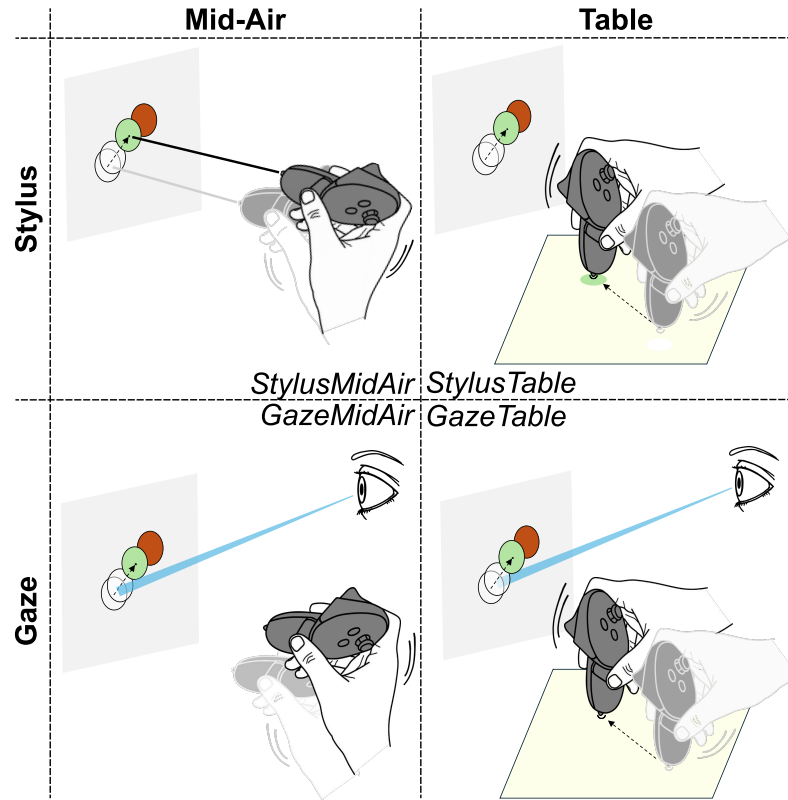


Figure 3.1: Illustration of the four input techniques.

3.4.1 StylusMidAir

StylusMidAir uses the controller-based stylus substitute as a mid-air ray-based input device. In this technique, the system casts a ray from the stylus ray origin in the direction of the attached pen tip. The user selects an object by pointing the ray at the target and pressing a button. When the ray intersects the target object, the object is highlighted to provide visual feedback, helping the user understand when selection is possible.

After a successful selection, the object enters a dragging state(3.2). During dragging, the object is moved based on the intersection point between the stylus ray and the manipulation plane. To avoid sudden jumps when dragging begins, the system preserves the offset between the ray hit point and the central position of the object. The object position is updated using interpolation, and the ray origin and pen tip positions are filtered to reduce high-frequency jitter during interaction.

This technique represents a stylus-only mid-air baseline. It allows users to quickly point at and manipulate virtual objects without relying on a physical surface. Therefore, *StylusMidAir* is suitable for fast selection and coarse object movement. However, because the user's hand is unsupported, the technique may be less stable during precise adjustment and may require more physical effort during prolonged use.

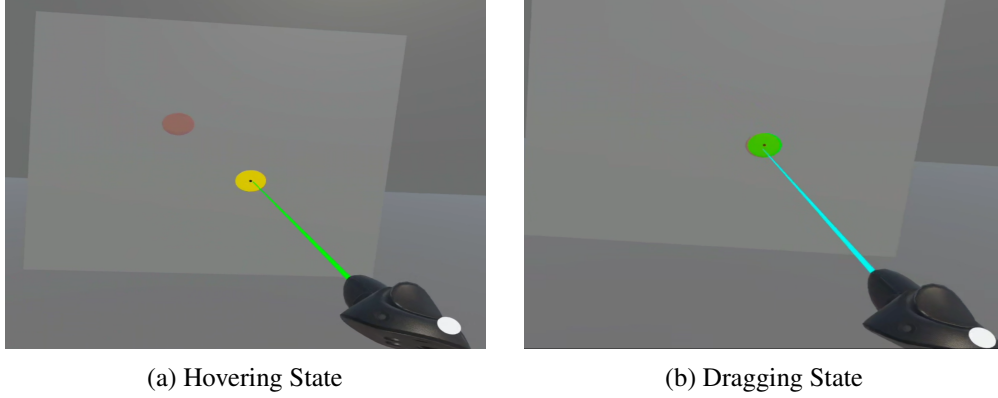


Figure 3.2: StylusMidAir interaction States

3.4.2 GazeMidAir

GazeMidAir combines gaze-based target selection with mid-air stylus movement. In this technique, the user's gaze is used to identify the intended target object. The system obtains the gaze ray from the eye-tracking component and checks whether the gaze ray intersects the target object. When the user is looking at the target, the object is highlighted to provide visual feedback. The user then confirms the selection by pressing a button.

After a successful selection, the object enters a dragging state(3.3). Unlike *Stylus-MidAir*, where the stylus ray is used for both pointing and manipulation, *GazeMidAir* separates target acquisition from object movement. Gaze is used for selecting the object, while the stylus controls the manipulation [8, 11, 20]. The system records the initial object position and the initial stylus position when dragging begins. During dragging, the object is moved according to the stylus movement relative to this starting position.

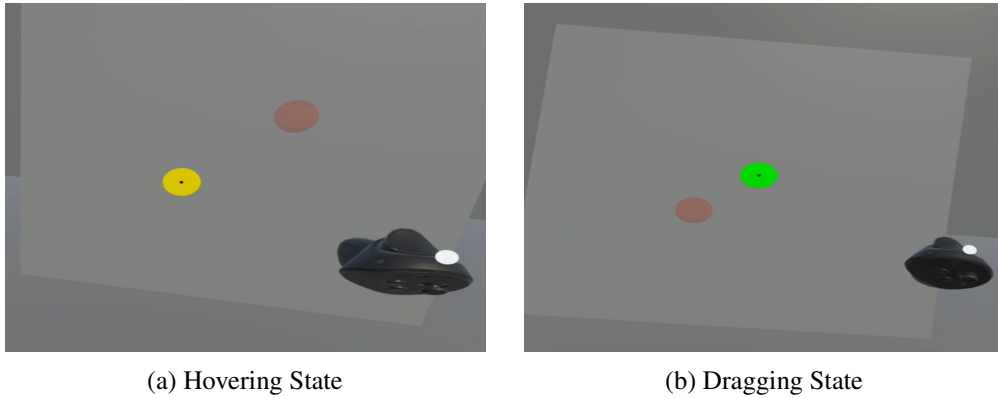


Figure 3.3: GazeMidAir interaction States

The movement is mapped using a distance-based control-display gain. This gain maps stylus motion to visual angle, so that a given angular movement of the stylus results in a corresponding angular movement of the object. In other words, the object follows the stylus' relative movement in visual angle rather than in raw physical distance. The object position is updated using interpolation.

This technique reduces the need for precise manual pointing because the hand does not need to aim a ray at the target object. Instead, the eyes provide target acquisition and the hand focuses on controlling movement. *GazeMidAir* therefore combines fast gaze-based selection with flexible mid-air manipulation. However, it may still be affected by eye-tracking accuracy, gaze timing, and the lack of physical support for the hand during manipulation.

3.4.3 StylusTable

StylusTable uses the stylus on a physical table surface for surface-supported manipulation. In this technique, the user's hand can rest on the table while moving the stylus, which provides physical support during interaction. The system tracks the position of the pen tip relative to the table and maps this movement to a cursor position on the virtual task plane. The input area on the table is defined as a square region based on the table size. In the current setup, this region is scaled by an area scale parameter, so only a smaller part of the physical table is used for controlling the virtual cursor.

The user selects the object by pressing the stylus against the table. Pressure input is used to determine whether the stylus is in contact with the surface. When the cursor is over the target object and sufficient pressure is detected, the object is selected. During dragging, the object follows the mapped cursor position while preserving the offset between the cursor and the object position. This helps avoid sudden jumps when manipulation begins. The object position is updated using interpolation. A short release debounce is also used so that brief pressure loss does not immediately cancel the interaction.

This technique represents a stylus-only surface-supported input approach. Unlike mid-air input, the user's hand is physically supported throughout the interaction, which can improve stability and comfort during precise movement. *StylusTable* is therefore suitable for fine positioning and alignment on the virtual task plane. However, because the input occurs on a physical surface while the object is displayed in XR, the technique relies on an input-output mapping between the table area and the virtual manipulation space [5, 6].

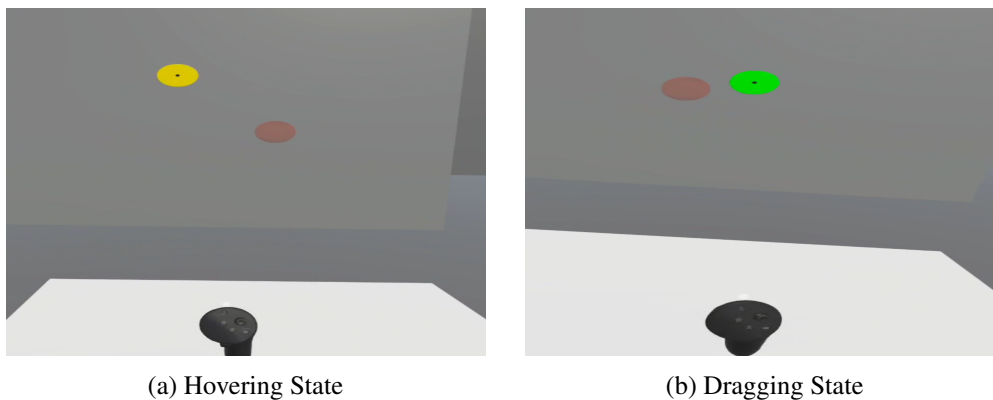


Figure 3.4: StylusTable interaction States

3.4.4 GazeTable

GazeTable combines gaze-based target selection with surface-supported stylus manipulation. In this technique, the user's gaze is used to identify the intended target object. The system obtains the gaze ray from the eye-tracking component and checks whether the gaze ray intersects the target object. When the user is looking at the target, the object is highlighted to provide visual feedback. The user confirms the selection by pressing the stylus against the physical table surface.

After a successful selection, the object enters a dragging state(3.5). *GazeTable* separates target acquisition from manipulation: gaze is used to select the object, while the stylus controls the object's movement on the table surface. When dragging begins, the system records both the initial object position and the initial stylus tip position. During dragging, the object is moved according to the stylus movement relative to this starting position.

The stylus movement is scaled using a control-display gain. In the current setup, this gain is set to 5, meaning that small movements of the stylus on the table are amplified when mapped to the virtual object. The object position is constrained within predefined movement boundaries and updated using interpolation. A short release debounce is also used.

This technique combines fast gaze-based target selection with stable surface-supported control. Because the hand rests on the table, *GazeTable* can reduce arm fatigue and support more precise manipulation than mid-air input. It also reduces the need for manual aiming, since the hand does not need to point directly at the target object. However, the technique may still be affected by eye-tracking accuracy, gaze timing, and the mapping between physical table movement and virtual object movement.

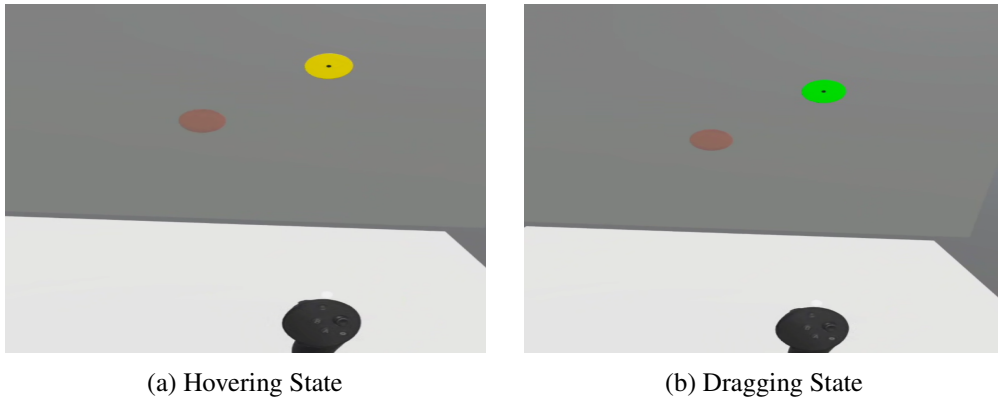


Figure 3.5: GazeTable interaction States

3.5 Input Space and Output Space

While the design space in Section 3.3 defines the four input techniques, this section focuses on how physical input is mapped to virtual object movement. This mapping is important because XR manipulation involves both a physical input space, where the user's hand and stylus move, and a virtual output space, where the object is displayed

and manipulated [5, 18, 19].

In the mid-air techniques, the physical input space and the virtual output space are closely related. The stylus moves in three-dimensional space, and its movement can be interpreted relative to the virtual object or the manipulation plane. This makes the control relationship relatively direct, although the user’s hand remains unsupported during manipulation.

In the table-based techniques, the physical input space and the virtual output space are more separated. The stylus moves on a two-dimensional physical surface, while the manipulated object is displayed in XR. The system must therefore map movement on the table to movement on the virtual task plane. This mapping determines how physical stylus movement corresponds to object movement, and it affects the user’s precision, control, and comfort.

This input-output relationship is central to the comparison between the four techniques. The mid-air techniques test interaction where input and output are spatially closer, while the table-based techniques test interaction where input is physically supported but indirectly mapped. By comparing these cases, the study examines how the relationship between input space and output space affects precise XR manipulation.

3.6 Application-Level Composition

The four techniques described in Section 3.4 are atomic: each one is a self-contained way of selecting and manipulating an object, and the user study of Chapter 4 evaluates each in isolation. The application of Chapter 5, however, runs in a hybrid mode in which exactly one of the four techniques (or one of the proxy-based variants developed in Section 3.8) is active at any given moment, and the choice is made automatically from three signals: the focused target’s tag (Interactable, Shadow, or none), the pen-tip pressure (in contact with the desk or in the air) and the controller trigger (pressed or released). The user never selects a technique by name. The empirical results of Chapter 4 should be read against this hybrid mode: the study isolates each atomic technique for measurement, while the application composes them into a single workflow.

3.7 Gaze-Mediated Axis Mapping

A 2D pen on a horizontal desk produces two input deltas per frame: a lateral delta (Δx) and a forward-backward delta (Δz). To use the pen for 3D manipulation, the system must decide on every frame which world axis the forward-backward delta corresponds to. The lateral delta is unambiguous - it always maps to the world X axis, parallel to the user’s shoulders - so the routing problem reduces to choosing a target for Δz .

The mapping is resolved from two cheap, continuously available signals: gaze pitch and gaze target. Gaze pitch has two states, Straight (pitch below 17°) and Down (pitch above 27°); the hysteresis band between the two thresholds prevents the routing from flickering as the user’s head sways. Gaze target also has two states, Object (the user is looking at the object being manipulated) and Pen (the user is looking down at the stylus, which activates the on-pen minimap proxy described in §3.6). The four combinations produce four routing rules:

- **Gaze Straight, Gaze on Object** - wall mapping: Δz routed to world Y (up). Pushing the pen forward across the desk lifts the object up the wall in front of the user.
- **Gaze Down, Gaze on Object** - floor mapping: Δz routed to world Z (away). Pushing the pen forward pushes the object away across the floor of the room.
- **Gaze Straight, Gaze on Pen** - top-down minimap: Δz routed to world Z, with the pen acting as a finger on a 2D map.
- **Gaze Down, Gaze on Pen** - vertical-plane minimap, switchable between front-facing XY and side-facing YZ views.

Two further mechanisms keep the routing usable in practice. The gaze-derived posture is locked at the start of each drag, so that natural head sway during manipulation does not change the mapping mid-motion. When the gaze target flips between Object and Pen mid-drag -the user looks down to consult the minimap - the system re-anchors the input deltas before applying the new routing, so that the object does not visually teleport at the moment the routing switches. Both mechanisms are described at the implementation level in §5.4.

The justification for tying the routing to posture is that gaze pitch is a reliable proxy for the user’s mental reference frame: when the user looks at a wall, the cognitive frame is vertical; when the user looks at the floor, it is horizontal. Aligning the input axes with the active reference frame removes the mental-rotation step that any fixed-mapping design imposes on the user.

3.8 Proxies

Direct surface input requires the user’s pen to be in contact with the part of the world they want to manipulate. In the application of Chapter 5, the things to be manipulated are 3D furniture items in a room-sized workspace, none of which the user can physically touch. Two proxies are introduced to bring distant content into reach without changing where the user is sitting.

The **shadow proxy** is a flat 2D handle, projected onto the desk beneath every interactable object. The shadow inherits the object’s XZ position frame by frame and is implemented as a trigger collider rather than a solid one, so that the pen tip and the gaze ray can both interact with it without colliding with the desk geometry. Pressing the pen onto a shadow grabs the linked object in Direct+2DOF mode, constrained to the floor plane of the room; gazing at the shadow and pressing the pen on the desk grabs the same object in Indirect+2DOF mode. The shadow turns the desk into an addressable surface for any object in the room, including objects floating in mid-air, at essentially no cost.

The **on-pen minimap proxy** is a wrist-anchored display, mounted on the controller, that shows a top-down (or, when the user is looking down, a front- or side-facing) orthographic view of the currently grabbed object and its surroundings. The minimap is invoked implicitly: when the system is already in an indirect manipulation state and the user looks down at the pen, the minimap activates and the axis routing switches to a coordinate system aligned with the minimap’s view. When the user looks back at the

object, the minimap shrinks and the routing reverts. The proxy gives the user a forward perspective for depth control without forcing them to physically walk around the object.

Chapter 4

Evaluation

This chapter presents the controlled user study used to evaluate the four input techniques introduced in Chapter 3. The evaluation compares gaze-assisted and stylus-only interaction across mid-air and table-based input spaces. The chapter first describes the evaluation goal, study design, task, participants, apparatus, data logging, procedure, measures, and statistical analysis. It then reports the results across performance, movement, and subjective experience measures.

4.1 Evaluation Goal

The goal of the evaluation is to compare gaze-assisted and stylus-only input techniques for precise plane-based manipulation in XR [4, 8, 11]. The study focuses on four input techniques: *StylusMidAir*, *GazeMidAir*, *StylusTable*, and *GazeTable*. These techniques vary along two main dimensions: target selection method and physical input space. Target selection is performed either by stylus pointing or by gaze, while manipulation is performed either in mid-air or on a physical table surface.

Although the broader interaction concept concerns XR object manipulation, the evaluation uses a controlled docking task to isolate fundamental input performance. The task is designed to evaluate plane-based manipulation rather than full 3D object manipulation [12, 13]. This boundary is important: the study does not claim to measure complete 3D manipulation performance, but instead focuses on controlled 2D manipulation within XR, where target acquisition, movement precision, and input stability can be compared consistently across techniques.

The evaluation aims to understand the trade-offs between speed, precision, physical effort, visual effort, comfort, and user preference. In particular, it examines how gaze-based selection affects target acquisition, how table-based input affects manipulation precision and comfort, and how mid-air input affects speed and stability.

This evaluation is designed to answer the research questions introduced in Chapter 1. RQ1 is addressed by comparing gaze-assisted and stylus-only selection through acquisition time, task completion time, and failed selections. RQ2 is addressed by comparing mid-air and table-based manipulation through positional error, stylus movement, stylus rotation, and physical comfort. RQ3 is addressed by analyzing the trade-offs across timing, accuracy, physical effort, eye fatigue, and user preference.

4.2 Study Design and Conditions

The study uses a within-subject design, where each participant completes the task using all four input techniques [11, 13]. This design allows the techniques to be compared directly within the same participant, reducing the influence of individual differences in XR experience, motor control, and familiarity with stylus input. The study therefore focuses on how performance changes across input techniques rather than between different participant groups.

The main independent variable is input technique, with four levels: *StylusMidAir*, *GazeMidAir*, *StylusTable*, and *GazeTable*. These four techniques form a 2×2 design based on two interaction dimensions. The first dimension is target selection method, which is either stylus-based selection or gaze-based selection. The second dimension is physical input space, which is either mid-air input or table-based input.

In addition to technique, the study varied three task factors: movement direction, movement distance, and target size. Movement direction had four diagonal directions. Movement distance had two levels: 10° and 20° visual angle. Target size had two levels: 4° and 6° . Each condition was repeated four times.

Factor	Levels
Technique	<i>StylusMidAir</i> , <i>GazeMidAir</i> , <i>StylusTable</i> , <i>GazeTable</i>
Movement direction	Four diagonal directions
Movement distance	10° , 20°
Target size	4° , 6°
Repetition	4 repetitions per condition

Table 4.1: Independent variables and levels in the user study.

Each participant completed all combinations of the study factors. In total, each participant completed 4 techniques $\times$ 4 movement directions $\times$ 2 distances $\times$ 2 target sizes $\times$ 4 repetitions, resulting in 256 trials per participant. This repeated-measures structure allowed the four techniques to be compared under the same experimental conditions.

4.3 Task Design

The task is a controlled docking task designed to evaluate fundamental plane-based manipulation in XR [12, 13]. In each trial, participants are shown two circular objects: an interactable circle and a target circle. The interactable circle is the object that participants can select and move, while the target circle indicates the desired final position. The goal is to move the interactable circle onto the target circle until the two visually overlap. The task focuses on plane-based docking so that the four techniques can be compared under the same controlled conditions.

The interactable circle and the target circle appear in a square layout. At the start of each trial, they are placed diagonally from each other, so participants need to move the interactable circle from one corner direction toward the opposite diagonal direction.

Four diagonal movement directions are used to avoid measuring only one movement direction and to reduce bias toward a particular hand or movement orientation.

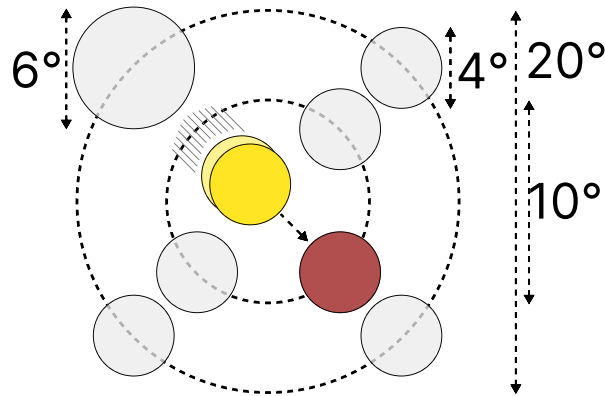


Figure 4.1: Task layout used in the docking task. Each trial contains a light gray interactable circle and a red target circle. The interactable circle turns yellow during interaction.

Movement distance is defined in visual angle rather than only in raw physical distance in the virtual scene [20, 24]. This makes the movement amplitude relative to the user's view. The two distance levels, 10° and 20°, create different movement amplitudes while keeping the task within a comfortable interaction range.

Target size is also defined in visual angle. The two target sizes, 4° and 6°, were selected to create two levels of docking difficulty: the 6° target provides a relatively easier condition, while the 4° target requires more precise control. These sizes are large enough to be clearly visible in XR, but different enough to reveal how each technique performs under different precision demands.

This task design allows the study to examine not only the overall differences between the four techniques, but also how those differences change with task difficulty. Larger distances may reveal differences in movement speed and physical effort, while smaller targets may reveal differences in precision and stability.

4.4 Participants

Twelve university students participated in the study. The participant group consisted of eight male and four female participants, and most participants were from Computer Science or related backgrounds. Eleven participants were between 18 and 25 years old, and one participant was between 26 and 34 years old.

All participants had normal or corrected-to-normal vision. Seven participants reported using glasses during daily life or visual tasks. Participants also differed in their previous experience with VR and AR systems. Three participants reported a high level of experience, four reported moderate experience, and five reported limited experience. This provided a participant group with mixed familiarity with immersive technologies.

4.5 Apparatus and Study Control

The study was conducted using a Unity-based XR prototype running on a Meta Quest Pro headset [9, 35]. The headset was used to display the virtual task environment and provide head tracking, controller tracking, and eye tracking. The prototype implemented the four input techniques described in Chapter 3: *StylusMidAir*, *GazeMidAir*, *StylusTable*, and *GazeTable*. The same task environment, virtual objects, headset, and input device were used across all conditions to keep the comparison consistent.

In the prototype implementation used for the study, stylus input was realized using a Meta Quest controller with an attached pen tip. The controller provided the tracked pose and button input, while the attached pen tip represented the stylus endpoint used for pointing or surface interaction. For simplicity, this controller-based input device is referred to as the “stylus” throughout the thesis. In mid-air conditions, participants held the stylus in space. In table-based conditions, participants used the pen tip on a physical table surface.

Participants completed the study while sitting in front of a physical table. The table served two functions. First, it provided physical support for the hand under table-based conditions. Second, it defined the physical input area for surface-based stylus movement. The prototype also included a virtual table. Before table-based conditions, the virtual table was calibrated to align with the physical table, so that surface-based stylus interaction corresponded to the real table position.

Several implementation choices were made to keep the comparison fair across techniques. The same visual task layout, target sizes, movement distances, headset, and stylus device were used in all conditions. Object highlighting was used consistently to indicate when selection was possible. For gaze-based techniques, gaze was used only for target selection, while manipulation was still controlled by stylus movement [8, 11, 20]. For table-based techniques, the same table calibration procedure and input area were used across participants. When necessary, the researcher used the left controller to recalibrate the virtual table position so that it remained aligned with the physical table.

The headset view was also cast to a laptop during the study. This allowed the researcher to observe the virtual scene from the participant’s perspective, monitor whether the task was displayed correctly, and check whether calibration or interaction problems occurred during the session. The cast view was used only for monitoring and study control, and participants still completed the task directly through the headset.

The experiment also included researcher-side keyboard controls. These controls allowed the researcher to select the active input technique, repeat training trials when necessary, move to the next block, and adjust the participant ID before data recording. Participants did not need to control these experimental states themselves. This helped keep the procedure consistent across participants and reduced the influence of non-task-related actions on the recorded performance measures.

4.6 Data Logging

The prototype recorded trial-level data during the study. Following the same general purpose as the data logging system described in previous XR evaluation work, the logging was designed to preserve the essential information needed to reconstruct each

trial and compute the objective measures used in the analysis. In this study, each row in the exported data corresponded to one trial and contained both experimental condition information and computed trial-level measures.

For each trial, the system recorded condition and identification variables, including participant ID, trial index, block index, training status, active input technique, movement direction, movement distance, visual angle, target size, and repetition index. These variables were used to identify the experimental condition of each trial and to separate training trials from the recorded trials used in the main analysis.

The system also recorded key event timestamps for each trial. These included the trial start timestamp, the timestamp of the first successful selection, and the trial end timestamp. Based on these event timestamps, the system computed acquisition time, manipulation time, and overall completion time. Acquisition time measured the time needed to successfully select the interactable object. Manipulation time measured the time from successful selection to the end of the trial. Completion time measured the total duration of the trial.

In addition to timing measures, the logged data included failed selection count, accumulative hand movement, accumulative hand rotation, final object position, target position, and positional error. Failed selection count recorded unsuccessful selection attempts before successful selection. Accumulative hand movement and accumulative hand rotation summarized the amount of physical input required during the trial. Final object position and target position were used to compute the final positional error after release.

The logged data was exported as CSV files for offline analysis. Training trials were identified using the training status variable and excluded from the main statistical analysis. The remaining recorded trials were used to compare the four input techniques across timing, error, movement, and rotation measures.

4.7 Procedure

At the beginning of the session, participants were introduced to the purpose of the study and completed the required consent and demographic forms. They were then seated in a stable position in front of the physical table and introduced to the current input technique. Before each technique, participants received a short explanation of how selection and manipulation worked and watched the instruction video at least once. Questions were clarified before the participant started the task.

Participants then put on the Meta Quest Pro and adjusted the headset for comfort, visibility, and stable eye tracking. For gaze-based techniques, eye-tracking calibration was completed before the task started, because gaze was used for target selection in *GazeMidAir* and *GazeTable*. For table-based techniques, the virtual table was aligned with the physical table before the recorded trials started.

After setup and calibration, participants completed at least one training block for the current technique. The training block allowed them to become familiar with the selection method, input space, and mapping between stylus movement and object movement before starting the recorded trials. If participants were still unsure about the technique, the researcher could repeat the training block before moving on to the recorded trials. Training trials were used only for practice and were not included in the

main analysis.

For gaze-based techniques, an auxiliary gaze ray was shown during training to help participants understand how gaze selection worked. This gaze visualization was hidden during the recorded trials to avoid changing the main task experience.

After training, participants completed the main docking task for the current technique. Each technique included trials in all combinations of movement direction, movement distance, target size, and repetition. The order of the four techniques was counterbalanced across participants to reduce ordering effects such as learning, fatigue, or increasing familiarity with the task. Participants were allowed to rest between techniques when needed.

After completing each technique, participants provided subjective feedback about their experience. This included ratings related to workload, physical comfort, eye fatigue, and preference. Participants were also encouraged to describe what felt easy, difficult, comfortable, or tiring about each technique. At the end of the study, participants gave additional qualitative comments comparing the four techniques.

4.8 Measures and Analysis

The evaluation collected both objective performance measures and subjective user experience measures [4, 36]. Objective measures were used to compare how efficiently and accurately participants completed the docking task with each technique. Subjective measures were used to understand how participants perceived workload, comfort, fatigue, and preference.

Research question	Main comparison	Measures
RQ1	Gaze vs. Stylus selection	Acquisition time, failed selections, task completion time
RQ2	Table vs. Mid-air input	Positional error, hand movement, hand rotation, comfort
RQ3	Overall trade-offs	Timing, error, movement, eye fatigue, preference

Table 4.2: Relationship between research questions, comparisons, and measures.

For performance, the study measured acquisition time, manipulation time, task completion time, failed selections, and positional error. Acquisition time was measured as the time from the start of a trial to the first successful selection of the interactable object. Manipulation time was measured as the time from the first successful selection to the end of the trial. Task completion time measured the total time required to complete one trial. Failed selections counted unsuccessful selection attempts before successful selection. Positional error measured the final distance between the interactable object and the target after release.

The study also measured hand movement and hand rotation during each trial. Hand movement was computed as the accumulated movement of the controller-based stylus, while hand rotation was computed as the accumulated rotation of the stylus during interaction. These measures were included to estimate the amount of physical input

required by each technique and to compare the physical demands of mid-air and table-based input.

Subjective measures were collected after each input technique. Participants rated their perceived workload, physical comfort, eye fatigue, and general preference. They were also asked to provide qualitative comments about the strengths and limitations of each technique. These comments were used to support the interpretation of the quantitative results and to identify usability issues that may not be fully captured by timing or error measures.

For statistical analysis, the distribution of each objective measure was first examined using Shapiro-Wilk tests [37]. Since the performance measures were not normally distributed, the analysis used Aligned Rank Transformation (ART) for a three-way repeated-measures ANOVA [38]. The three factors were input technique, movement distance, and target size. Holm-Bonferroni correction was applied to post-hoc comparisons to reduce the risk of false positives caused by multiple comparisons [39]. Outliers were filtered based on overall task completion time using the mean $\pm$ 2 standard deviations criterion.

Subjective data were analyzed separately. Friedman tests were used for subjective ratings, followed by post-hoc Wilcoxon tests where appropriate [40,41]. Holm-Bonferroni correction was also applied to these post-hoc comparisons.

4.9 Results

This section reports the results of the user study across objective performance measures, movement measures, and subjective ratings. The significant ART repeated-measures ANOVA results are shown in Table 4.3. The main effects of technique are visualized separately in Figures 4.2, 4.3, and 4.4. The results are organized into three parts: performance metrics, movement and rotation, and subjective ratings. Each part is linked back to the research questions introduced in Chapter 1.

Table 4.3: Significant ART repeated-measures ANOVA results across objective performance, error, movement, and rotation measures. Columns indicate the reported degrees of freedom (df), F score, p value, and partial eta squared η_p^2 . Here, T is Technique, D is Distance, and S is Size.

<i>Effect</i>	<i>df</i>	<i>F</i>	<i>p</i>	η_p^2	<i>Effect</i>	<i>df</i>	<i>F</i>	<i>p</i>	η_p^2
Acquisition Time					Failed Selections				
<i>T</i>	3,165	9.594	<.001	.149	<i>D</i>	1,165	11.790	<.001	.067
<i>D</i>	1,165	53.553	<.001	.245	<i>S</i>	1,165	7.495	.007	.043
<i>S</i>	1,165	32.787	<.001	.166	<i>T</i> $\times$ <i>S</i>	3,165	3.378	.020	.058
Manipulation Time					Positional Error				
<i>T</i>	3,165	11.085	<.001	.168	<i>T</i>	3,165	25.672	<.001	.318
<i>D</i>	1,165	54.737	<.001	.249	<i>D</i>	1,165	8.998	.003	.052
<i>S</i>	1,165	5.581	.019	.033					
Task Completion Time					Hand Movement				
<i>T</i>	3,165	14.838	<.001	.212	<i>T</i>	3,165	141.301	<.001	.720
<i>D</i>	1,165	70.025	<.001	.298	<i>D</i>	1,165	408.843	<.001	.712
<i>S</i>	1,165	19.931	<.001	.108	<i>T</i> $\times$ <i>D</i>	3,165	22.445	<.001	.290
Hand Rotation									
<i>T</i>	3,165	51.019	<.001	.481					
<i>D</i>	1,165	122.379	<.001	.426					

4.9.1 Performance Metrics

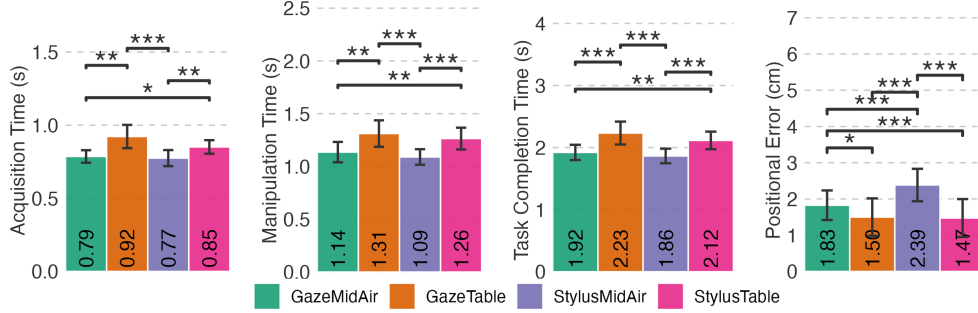


Figure 4.2: Main effects of Technique on performance measures. Bars show means with 95% confidence intervals. Significance is annotated as * for $p < .05$, ** for $p < .01$, and *** for $p < .001$.

The performance metrics show clear differences between the four input techniques. Overall, the MidAir techniques were faster than the Table techniques across the timing measures. Significant effects of technique were found for acquisition time, manipulation time, and task completion time, indicating that the choice of input technique affected how quickly participants could complete the docking task.

For acquisition time, *GazeMidAir* showed the shortest mean acquisition time, followed closely by *StylusMidAir*. The Table techniques required longer acquisition times, with *StylusTable* showing the slowest mean value. This indicates that target acquisition was generally faster when the interaction was performed in mid-air. Gaze-based selection also supported fast acquisition, especially in the mid-air condition.

For manipulation time, *StylusMidAir* showed the shortest mean manipulation time, followed by *GazeMidAir*. Both Table techniques required longer manipulation times. This indicates that table-based manipulation introduced additional time costs compared with mid-air manipulation.

Task completion time followed the same general pattern. *StylusMidAir* and *GazeMidAir* were faster than *StylusTable* and *GazeTable*. The statistical results show that the MidAir variants were significantly faster than the Table variants across the relevant timing comparisons, with all reported post-hoc comparisons reaching significance ($p \leq .022$). This indicates that mid-air input supported faster overall task execution.

The positional error results showed the opposite pattern. The Table techniques produced lower final positional error than the MidAir techniques, with significant differences across relevant comparisons ($p \leq .02$). *StylusTable* and *GazeTable* showed the lowest mean positional errors, while *StylusMidAir* produced the highest mean positional error. *GazeMidAir* also showed significantly lower positional error than *StylusMidAir* ($p < .001$). This shows that gaze-assisted selection in the mid-air condition was associated with lower final positional error than stylus-only mid-air input.

Task difficulty also affected performance. The shorter 10° movement distance resulted in faster performance than the longer 20° distance across the timing measures ($p < .001$). The shorter distance also produced fewer failed selections ($p < .001$) and lower positional error ($p = .003$). Target size also affected performance. The smaller 4° target size led to slower performance across timing measures ($p < .001$) and more

failed selections ($p = .007$) than the larger 6° target size.

The failed selection results should be interpreted with caution because an interaction effect was found between technique and target size. In particular, the comparison between *GazeTable* and *StylusTable* showed a Technique $\times$ Size interaction ($p = .027$). This indicates that the effect of target size was not identical across all techniques and that gaze-based selection in the table condition may have been more sensitive to smaller targets.

Together, these performance results provide evidence for RQ1 and RQ2. For RQ1, the results show that gaze-assisted selection supported fast acquisition in the mid-air condition and reduced positional error compared with stylus-only mid-air pointing. For RQ2, the results show that table-based input produced lower positional error but required longer task times. These patterns are discussed further in Chapter 6.

4.9.2 Movement and Rotation

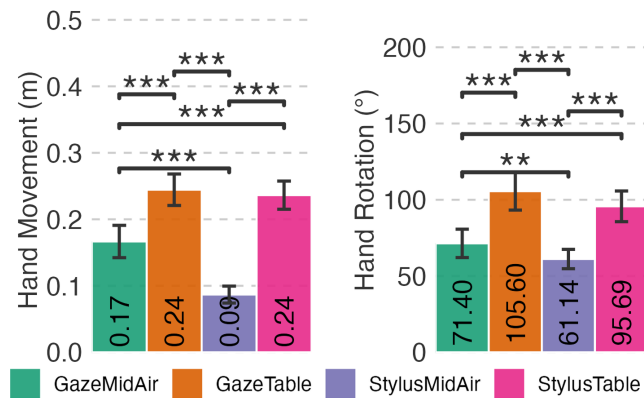


Figure 4.3: Main effects of Technique on hand movement and hand rotation. Bars show means with 95% confidence intervals. Significance is annotated as * for $p < .05$, ** for $p < .01$, and *** for $p < .001$.

The movement and rotation measures show that the four techniques required different amounts of physical input. *StylusMidAir* produced the lowest hand movement and hand rotation overall. This result is consistent with its ray-based interaction style, where participants could select and manipulate objects using relatively small physical movements. The differences between *StylusMidAir* and the other techniques were significant for movement and rotation measures, with all relevant comparisons reaching significance ($p < .008$).

GazeMidAir required more movement and rotation than *StylusMidAir*, but less than both Table techniques. The differences between *GazeMidAir* and the Table techniques were significant for movement and rotation ($p < .001$). This indicates that gaze-assisted mid-air input required less physical movement than the table-based techniques.

The Table techniques required larger hand movement and hand rotation than the MidAir techniques. This may be related to the constrained input area on the physical table surface, where participants sometimes needed to reposition the stylus during

manipulation. These higher movement and rotation values occurred together with lower positional error for the table-based techniques.

Movement distance also affected hand movement and hand rotation. The shorter 10° movement distance resulted in less hand movement ($p < .001$) and less hand rotation ($p < .001$) than the longer 20° distance. Hand movement was also affected by an interaction between technique and movement distance. *StylusMidAir* was less affected by the longer movement distance than the other techniques, with all relevant comparisons reaching significance ($p < .001$). *GazeMidAir* was also less affected by longer distance than *GazeTable* ($p = .028$). This shows that increasing the movement distance did not affect all techniques in the same way.

These movement results provide evidence for RQ2 and RQ3. Surface-supported techniques did not reduce physical input movement in this task; instead, they required more hand movement and rotation than the mid-air techniques. At the same time, the table-based techniques produced lower positional error. The implications of this relationship between physical movement and precision are discussed in Chapter 6.

4.9.3 Subjective Ratings

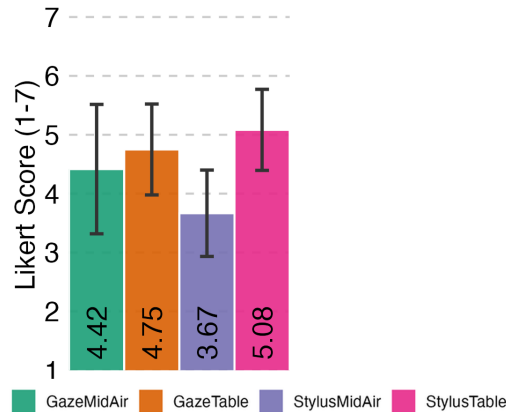


Figure 4.4: Main effect of Technique on preference score. Bars show means with 95% confidence intervals.

For subjective workload, the NASA-TLX measures did not show clear differences between the four techniques. This suggests that participants did not report large workload differences across conditions, even though the techniques differed in timing, accuracy, movement, and rotation.

Eye fatigue showed a clearer difference. The Friedman test indicated a significant difference in perceived eye fatigue across techniques ($\chi^2(12) = 25.436$, $p < .001$, $W = .983$). Participants reported less eye fatigue for the stylus-only techniques than for the gaze-based techniques, with significant post-hoc comparisons reported across the relevant pairs ($p \leq .023$). This indicates that using gaze as an active input channel increased visual effort.

Preference ratings did not show a statistically significant difference between techniques. However, the mean ratings and participant comments still revealed useful

trends. Table-based techniques were often described as physically comfortable because participants could rest their arm on the table. Mid-air techniques were often described as faster, but several participants found them tiring during repeated trials. Gaze-based selection was generally considered intuitive, but some participants also reported timing issues when their gaze moved away from the target before the confirmation action was completed.

These subjective results provide additional evidence for RQ3. Gaze-based techniques were often perceived as intuitive, but they were also associated with higher eye fatigue. Table-based techniques were often perceived as physically comfortable because the hand could rest on the table, even though they were slower and required more hand movement. These findings show that subjective experience did not follow a single objective measure such as speed or positional error.

4.9.4 Summary

In summary, the results show consistent differences between the four input techniques. The MidAir techniques were faster than the Table techniques, while the Table techniques produced lower positional error. *StylusMidAir* required the least hand movement and hand rotation, whereas the Table techniques required more physical input. *GazeMidAir* showed lower positional error than *StylusMidAir*. However, gaze-based techniques were associated with higher eye fatigue.

Overall, the results show that the four techniques produced different strengths and limitations across the measured variables. Mid-air input supported faster task execution, table-based input supported more precise final placement, gaze-assisted input supported fast and intuitive target selection, and stylus-only input avoided additional visual effort. These findings provide the basis for the discussion of speed, precision, physical effort, visual effort, and user preference in the following chapter.

Chapter 5

Application Design

This chapter describes the design and implementation of a hybrid VR interior-design application that realizes the conceptual model developed in Chapter 3. The overarching contribution is not any single technique, but the demonstration that the design space characterized by the four input techniques studied in Chapter 4 - selection method crossed with input space - can be instantiated as a single continuous workflow on commodity XR hardware, with implicit transitions between modes and without explicit mode UI. The application’s manipulation modes are inspired by the study’s four techniques but adapt them to a 3D scene rather than a 2D panel, and add one mode - gaze plus pen-on-desk with gaze-driven axis routing - that the controlled study could not directly evaluate. The chapter proceeds from the scenario and apparatus (§5.1), through the system architecture and interaction state machine (§5.2), the per-state manipulation behaviors (§5.3), the gaze-dependent axis routing (§5.4), and the proxy and feedback subsystems (§5.5), to limitations (§5.6), development reflections (§5.7), and a closing outlook on the application’s two open design problems (§5.8).

5.1 Scenario, Motivation, and Hardware

Why interior design as the application scenario

The controlled study of Chapter 4 isolates four atomic input techniques on a simplified plane-based task. To understand how those techniques behave in a realistic workflow—and to expose the design problems that a plane-based task cannot reveal—an application context is needed. We chose interior design for three reasons. First, it is the canonical XR object-manipulation scenario: spawning, moving, rotating, and aligning virtual objects inside a room-sized scene is the use case that consumer applications such as IKEA Place and Gravity Sketch have built around, and that prior research systems such as SymbiosisSketch [15] and VRSketchIn [29] have explicitly addressed. Second, it stresses both coarse placement (dropping a sofa into a room) and fine alignment (sliding it flush against a wall) within the same session, which forces the system to span the full range of input modalities rather than optimizing for one of them. Third, the scenario is recognizable to non-expert users; informal testers reach competence without an extensive tutorial, which keeps the implementation work focused on the interaction model rather than on scenario-specific training. Having fixed the scenario, the next

question is what input model to use within it.

Why a hybrid surface-air pen, and not a single-mode design

A pure mid-air design - 6-DOF stylus, controller-tracked, no surface - is the default in most VR authoring tools, and it is what the *StylusMidAir* condition of Chapter 4 measures. It supports rapid 3D placement well, but it is the worst case for precise alignment: hand tremor and *gorilla arm* fatigue dominate as soon as the task requires sub-centimeter accuracy, and the trigger press triggers the Heisenberg effect, causing a slight movement that pulls the object off target at the moment of release. [42]. A pure surface design, stylus pressed onto a tracked tablet or desk, inverts these strengths: it is stable and comfortable for fine work, but it cannot reach distant content without an explicit proxy gesture and is poorly suited to coarse 3D placement.

The application takes a different position: keep a single stylus, keep the surface fixed (an ordinary desk), and let the same pen tip serve both roles. The pen acts as a 6-DOF mid-air tool when held in the air and as a desk-supported precision tool when pressed onto the desk; the system decides which role is active from continuously available signals (pen-tip pressure, gaze target, gaze-ray angle). The non-dominant hand is freed for a wrist-anchored tool menu rather than for holding a second surface (tablet), and the alignment between the desk and the scene is recomputed every frame from the user's gaze. The remainder of this section describes the apparatus that realizes this design, and the following sections develop each part of the model in turn.

Apparatus and hardware

The application is a hybrid VR interior-design tool.¹ The user sits at an ordinary desk, wears a Meta Quest Pro headset, and holds a single Touch Pro controller in an inverted "stylus" grip. In front of the user is a 2 m × 2 m virtual room with a configurable set of furniture prefabs (sofa, chair, lamp, table, and so on). Using a wrist-anchored tool menu (Figure 5.1) on the non-dominant controller, the user can spawn furniture, delete it, duplicate it, recolor it, or-most often-move and rotate it. Figure 5.2 shows the resulting workflow across the four interaction primitives. The same physical desk that supports the user's hand serves as the pen's input surface; the same pen tip serves both for mid-air interaction (when held in the air) and for surface interaction (when pressed down on the desk).

Within the design space of Chapter 3, the application instantiates four coexisting interaction primitives - Direct+3DOF, Direct+2DOF, Indirect+2DOF, and Proxy-Direct+2DOF - with implicit transitions between them.

The hardware platform is the Meta Quest Pro running the Meta XR All-in-One SDK on Unity 6. The Touch Pro controller's stylus tip exposes a pressure-sensitive force axis (`OVRInput.Axis1D.PrimaryStylusForce`) that we use as the desk-contact signal; the trigger is read separately for mid-air engagement. Eye tracking is provided by the integrated `OVREyeGaze` API; we wrap the raw eye-gaze samples in a One Euro filter [43] to suppress jitter without introducing perceptible lag. No external trackers and no second device are required.

¹Source code and a demonstration video are available at <https://github.com/platakism/MScThesis> and <https://youtu.be/bcVJDniHlLo>.



Figure 5.1: *Tool Menu* to spawn a prefab: (a) From the main menu select the Spawn Prefab option (using gaze and trigger button). (b) From the prefab menu select the prefab you want to spawn in the room. Then press the trigger button on right controller to spawn the prefab on the floor at the XZ position of your right controller.

5.2 System Architecture and Interaction State Machine

Why a finite state machine, and why these six states

The behavior the user perceives is deceptively simple - "the pen does the right thing depending on what I am looking at and whether I am touching the desk" - but the underlying logic must distinguish at least five mutually exclusive manipulation modes plus an idle state. We chose an explicit finite state machine over the two alternatives we considered, and the reasoning is worth recording.

A naive event-driven design, in which each input transition fires handlers without an explicit state variable, was attractive because it requires no central dispatcher. We rejected it because the system must enforce strict mutual exclusion (the user cannot simultaneously direct-drag and indirect-drag), and event-driven designs encode that mutual exclusion implicitly across many handlers, which is hard to reason about and harder to test. A behavior-tree design was also attractive because it composes well; we rejected it as overkill for the small number of leaf behaviors and because behavior trees are awkward at expressing the "hold this state while a continuous signal is true" pattern that our manipulation modes need.

An explicit FSM, on the contrary, makes mutual exclusion structural (only one state is active), gives a single Update loop to inspect when debugging, and makes the gating conditions visible as a transition table rather than scattered across listeners. The six states - Idle, DirectAir, DirectTableShadow, IndirectAir, IndirectTableShadow, IndirectTableObject - are the minimum partition of the design space that distinguishes the five manipulation behaviors the application offers. Collapsing any pair (for example, treating DirectAir and DirectTableShadow as a single state with branching inside the handler) would re-introduce inside the handler the implicit branching the FSM was designed to make explicit. The next subsection lays out the six states concretely, the transition conditions, and the class decomposition that implements them.

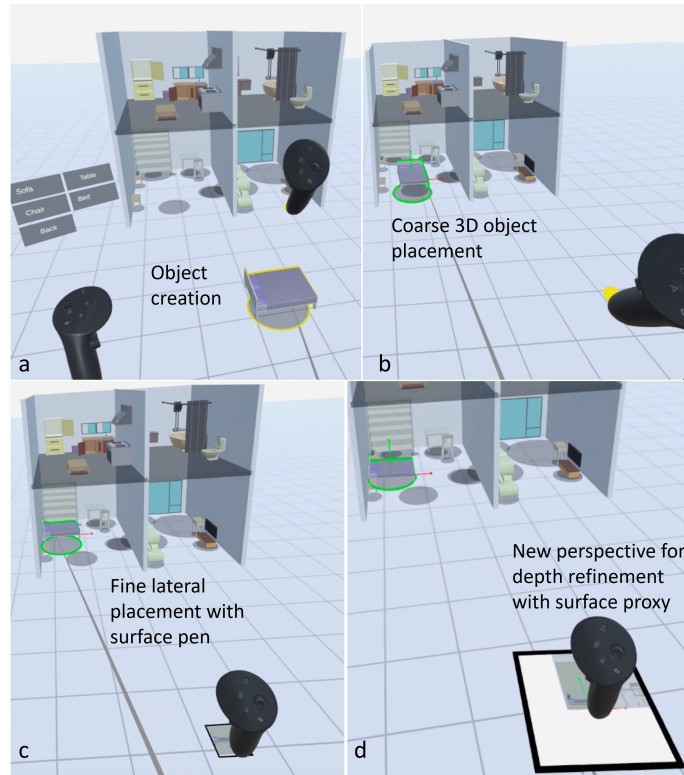


Figure 5.2: *Interior Design* workflow demonstrating multi-DOF object manipulation: (a) object creation via gaze and mid-air input; (b) coarse 3DOF placement; (c) fine lateral adjustment using a surface pen; (d) depth refinement through a secondary perspective.

State machine details

The state machine has six different states (Figure 5.3). Idle is the default; in Idle, the system performs targeting (decides what the user is currently focused on) and applies a yellow hover outline, but does not move anything. Any of the five active states is entered when three signals align: a trigger or pressure event, a focused target with a particular tag (Interactable or Shadow), and the type of surface contact (touching the object, gazing at the object, or gazing at the shadow). Each active state holds for as long as its gating input is held; releasing the trigger or releasing pressure on the desk returns the system to Idle. There is no other way out of an active state, and no explicit way to switch between active states without first returning to Idle.

The class decomposition follows the FSM (Figure 5.4). HybridPenController owns the state variable, the per-frame Update() loop, and the dispatch to three handler classes. PenTargetingHandler resolves the focused target each frame from two sources of evidence: a small Physics.OverlapSphere around the pen tip (for proximity-based direct targeting) and a Physics.SphereCastAll along the gaze direction (for indirect targeting). When both an Interactable and its associated Shadow are hit simultaneously—the common case when the pen is hovering over an object that has its shadow on the desk—a tie-breaker uses pen-tip pressure: if the pen is pressed down, the Shadow wins (the user’s intent is to slide on the desk); if it is in the air, the Interactable wins. PenMovementHandler applies the per-state movement deltas; it is the largest class

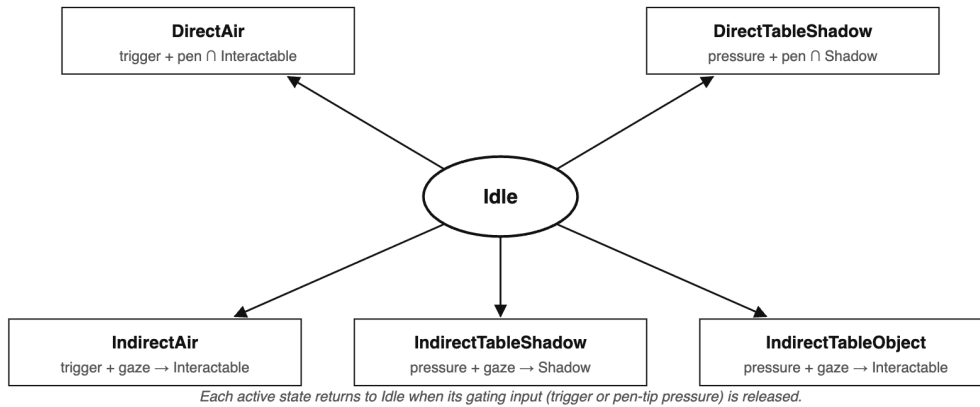


Figure 5.3: **Six-state interaction FSM.** Idle is the default; the five active states are entered when a trigger or pen-tip-pressure event combines with a focused target of the indicated tag. Each active state returns to Idle when its gating input is released; there is no other transition path between active states.

because it owns the gaze-dependent axis routing of §5.4. PenInteractionHandler handles the start- and end-of-drag bookkeeping: capturing the grab offset between pen tip and object center, capturing the grab rotation offset, and locking the gaze-derived posture for the duration of the drag. Beyond the three handlers, the application uses a small number of supporting components. GazeProvider wraps the OVREyeGaze API and exposes a smoothed gaze ray plus a 300 ms history buffer used by the sticky-gaze logic of §5.3.3. ShadowManager walks the scene at start-up and spawns one ShadowFollower for every object tagged *Interactable*; ShadowFollower snaps its transform to the object’s XZ position at a fixed Y (the table height) every frame. InteractionToolManager is a singleton that holds the current tool (Move, Delete, Duplicate, ColorPicker, Spawn) and the current sub-menu visibility. TableAutoCalibrator lets the user re-set the desk plane height at runtime by holding the A button and pressing the pen against the real desk; this accommodates desks of varying height without an external setup step. The visual subsystems (AxesFeedbackManager, HandMenuController, ControllerShadowGrid, TableGridGenerator, GazeRayVisualizer, ControllerOverlay) are described in §5.5.

What we learned the hard way

The current architecture was not the application’s first design. An earlier version held all of the targeting, movement, outline, and interaction logic in a single HybridPenController class. As the feature set grew, the class became unmanageable: physics-heavy code (raycasts, overlap spheres) and cosmetic code (outline material assignment) were interleaved frame by frame, and small changes to the visual feedback risked shifting the targeting logic. The current decomposition isolates the three concerns and makes per-frame behavior visibly testable. We record this not as a cautionary tale but as a design constraint: any extension to the system - a new tool, a new visual cue, a new proxy mode - should be addable without modifying HybridPenController, and the three-handler split is what makes that possible.

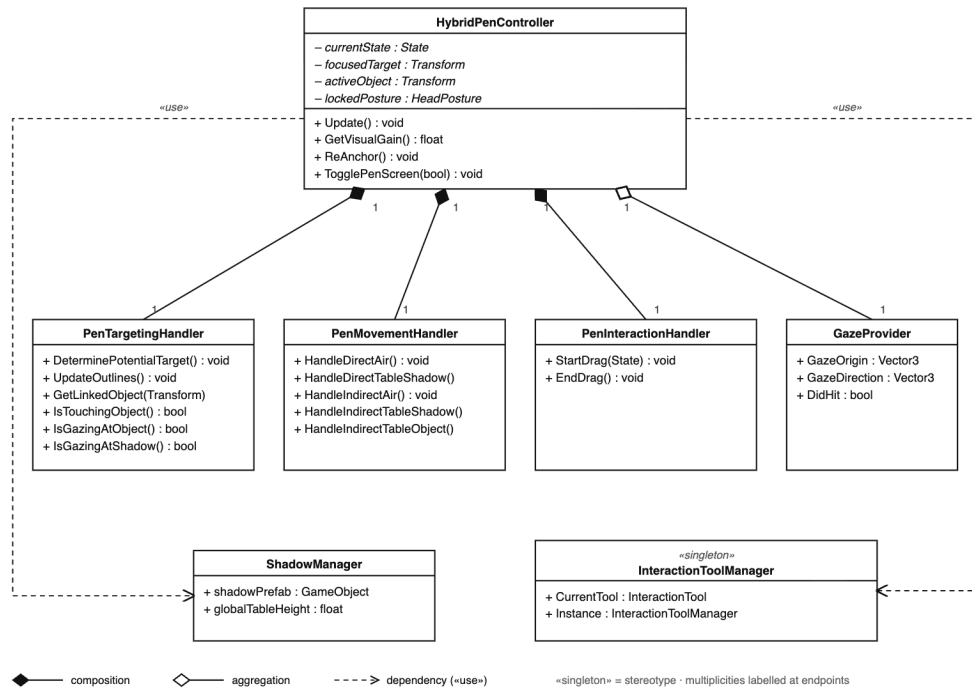


Figure 5.4: **UML class diagram of the manipulation subsystem.** HybridPenController is composed of three handler classes that split targeting, movement, and interaction concerns (filled-diamond composition). GazeProvider is aggregated (open diamond). ShadowManager and InteractionToolManager are reached as dependencies.

5.3 Direct and Indirect Manipulation

Why support both direct and indirect modes

A purely direct interaction model - grab what your hand is touching - is intuitive but caps the working volume at arm's reach. In a 2 m × 2 m virtual room operated from a single seated position, most of the scene is outside that volume. A purely indirect model - always manipulate via a ray or via gaze - is flexible at distance but feels detached for nearby objects, where the user's instinct is to simply reach for them. The application supports both modes and lets the system choose: physical proximity (within 5 cm of the pen tip) triggers direct manipulation; everything else routes through gaze. This matches user expectation without requiring a mode menu. Within both modes, however, the engagement gesture differs in a way that deserves its own justification.

Why pressure on the desk, but trigger in the air

The engagement gesture differs between direct and indirect modes for a specific reason: the Heisenberg effect [42]. Pressing the controller trigger involuntarily displaces the controller's pose by several millimeters, which is enough to pull a target off center at typical XR distances. In mid-air, the trigger is the only practical engagement signal, and the resulting perturbation is absorbed by the surrounding visual-angle gain. On the desk, however, pressing the pen into the desk uses the desk to absorb the press -

the pen physically cannot move while it is pressed down - so pressure is the strictly more accurate engagement signal whenever a surface is available. We use whichever is appropriate for the mode: trigger for air-states, pressure for desk-states. With the engagement rules in place, we can now describe what each of the five active states actually does to the manipulated object.

Per-state behaviour

Five of the six FSM states are manipulation states. **Direct+3DOF** (DirectAir) is entered when the user presses the controller trigger while the pen tip is overlapping an Interactable in mid-air. The handler captures the offset between the pen tip and the object's pivot at the moment of engagement (`grabOffset`) and the relative rotation between them (`grabRotationOffset`). In each subsequent frame, the object's pose is set to `penTip.position + grabOffset` and `penTip.rotation × grabRotationOffset`. The use of relative offsets rather than absolute positioning is what makes the interaction feel like sliding rather than snapping-the object stays exactly where the user grabbed it, regardless of where on the object the pen tip happened to be. **Direct+2DOF** (DirectTableShadow) is entered when the user presses the pen tip onto an object's shadow on the desk. The same offset logic applies, but the *Y* axis of the object is preserved: the user can slide the object across the floor of the room (*XZ*) but cannot lift it off the floor by accident. This matches the user's mental model of a shadow as a floor-bound handle (Figure 5.5). **Indirect+3DOF** (IndirectAir) is entered when the user presses the trigger while their

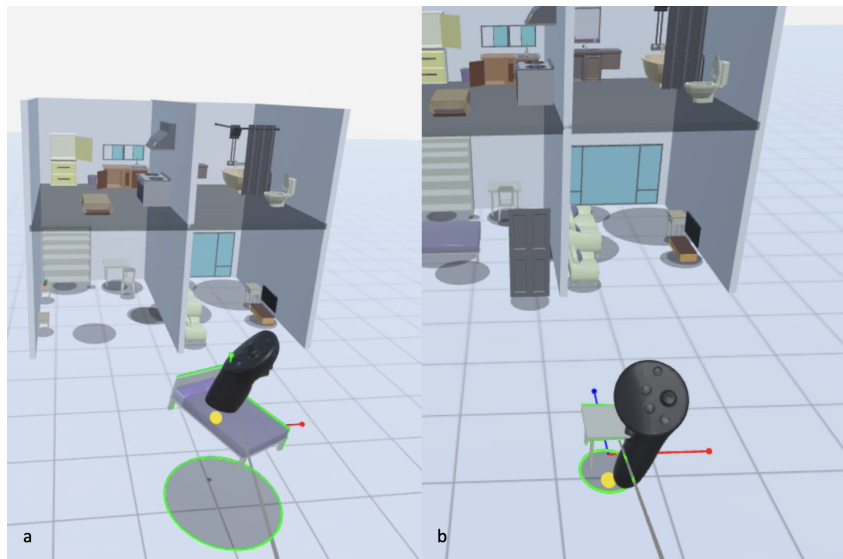


Figure 5.5: Direct manipulation modes. (a) DirectAir: the pen tip overlaps a nearby bed and the trigger engages a 1:1 6-DOF drag. (b) DirectTableShadow: the pen is pressed onto the table's shadow on the desk, constraining motion to the *XZ* floor plane.

gaze is fixed on a distant Interactable, with the pen tip not in physical contact with the object. The system applies the visual-angle gain of §5.3.1 so that the object follows the pen with a scaled motion - small hand movements produce large object movements at distance. This mode handles coarse 3D placement: dropping a chair into a far corner of

the room without standing up. **Indirect+2DOF** on a shadow (IndirectTableShadow) and on an object (IndirectTableObject) both engage when the user presses the pen onto the desk while gazing at, respectively, a distant shadow or a distant object. The first is a planar slide; the second is the central case for which the gaze-and-posture axis routing of §5.4 was designed (Figure 5.6).

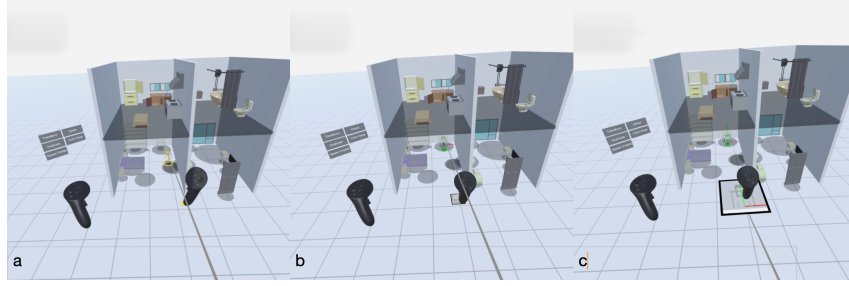


Figure 5.6: Indirect-On-Table manipulation. (a) Acquire target (chair). (b) XZ manipulation while looking at object. (c) XY manipulation while looking at minimap.

5.3.1 Visual-angle gain

In an interior-design workflow, the user manipulates objects at distances ranging from arm’s reach (an immediate neighbor) to the far wall of the room (a couch in the corner). The application uses a continuously evaluated visual-angle gain:

$$gain = \frac{\|eye - object\|}{\|eye - pen\|} \quad (5.1)$$

The angular motion of the object as seen from the user’s eye is proportional to the angular motion of the pen as seen from the same eye. A user manipulating an object 2 m away with the pen 0.5 m from their face experiences a $4\times$ motion gain; if the user stands up and brings the pen closer to their face, the gain drops automatically. Casiez et al. [23] showed that adaptive gain consistently outperforms a fixed gain across pointing distances, and our visual-angle formulation is the version of that principle that requires no calibration and no per-task tuning. A small constant tableIndirectGainMultiplier of 1.30 is layered on top for the table-pressing modes, compensating for the slight under-reach we observed when users press into a flat desk and lose a few millimeters of motion to the press itself.

5.3.2 Re-anchoring on gaze shifts

Without re-anchoring, an axis-routing switch mid-drag would cause the object to teleport: the existing pen-from-anchor delta would be re-interpreted against a different world axis and the object would jump to a new position in a single frame. The indirect-table-object handler detects when the user’s gaze flips between Object and Pen mid-drag and, on every flip, calls ReAnchor(), which resets startPenPos and startObjPos to the current pen and object positions. Conceptually this is the same primitive as a mouse pickup, but driven by gaze rather than by lifting the device.

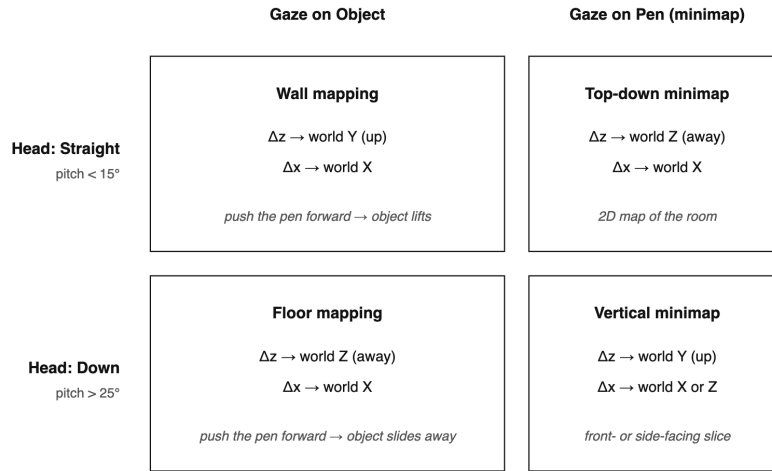
5.3.3 Sticky-gaze buffer

In a $2\text{ m} \times 2\text{ m}$ virtual workspace with eye-tracker accuracy of approximately $1.5^\circ\text{--}3^\circ$, natural ocular micro-jitter is large enough that an acquired distant target can be lost in the half-second between the user's intent forming and the user's pen pressing the desk. Without a buffer, the system reads this micro-jitter as a deliberate gaze shift and cancels the selection right before engagement; users describe this as "it always slips at the last moment". To handle this, GazeProvider exposes a 300 ms sticky-gaze buffer: once a target has been continuously gazed at for the buffer duration, it remains "acquired" for an additional grace period even after the gaze drifts off. The grace period is enough to absorb a single saccade and a deliberate downward glance to the desk; longer, intentional gaze shifts terminate the acquisition. The buffer is the difference between a system that feels stable at distance and one that feels twitchy.

5.4 Gaze-Dependent Axis Routing

Why axis routing is needed at all

A 2D pen on a horizontal desk produces two input deltas per frame: a lateral delta Δx and a forward-backward delta Δz . A 3D scene requires at least three. The missing axis has to come from somewhere. We considered three options. Fixing the mapping ahead of time (e.g. always route Δz to world Y) makes half of the workflow awkward: pushing the pen forward to slide a chair across the floor would lift the chair into the air instead. Adding an explicit toggle (e.g. a button that swaps between XY and XZ planes) works but imposes a mode-memory load on the user: every drag begins with the user recalling which plane the system is currently mapped to. Adding a second tracked surface (the SymbiosisSketch / VRSketchIn approach) decouples the mapping but adds a held device that occupies the non-dominant hand. The application uses neither approach. It routes the missing axis from two signals the user already produces during the task: gaze-ray angle and gaze target. These signals are continuously available on consumer XR hardware, cost no extra input action, and correlate strongly with the user's intended reference frame. When the user is looking at a wall, the cognitive frame is vertical; when the user is looking at the floor, it is horizontal. Aligning the input axes with the active reference frame removes the mental-rotation step that any fixed-mapping or toggle-based design imposes. An earlier version of the routing read the headset's pitch directly: when the head was tilted down past a threshold, the floor mapping engaged. That signal turned out to be the wrong one. Users routinely keep their head roughly straight while glancing down with their eyes at a piece of furniture on the floor, and the head-pitch reading would stay above the threshold even though the user's frame of reference had already shifted. The signal we actually want is where the user is *looking*, not where the headset is *pointing*, and that signal is the vertical angle of the gaze ray. The current implementation reads the gaze ray's pitch every frame, classifies it as Straight or Down through the hysteresis band of §5.7, and uses that classification together with the gaze target as the routing input.



Posture is locked at drag start; a gaze flip mid-drag triggers ReAnchor() so the object never teleports.

Figure 5.7: **Gaze-dependent axis routing.** A 2D pen delta on the desk (Δx lateral, Δz forward) is projected onto a different world plane depending on gaze pitch and gaze target. The lateral delta always maps to world X ; the forward delta is routed to either world Y or world Z according to the rules above. Gaze pitch is locked at drag start, and a gaze flip mid-drag triggers ReAnchor() so that the object does not visually teleport when the mapping changes.

Routing details

This section is the implementation companion of §3.5. The conceptual model - four routing rules selected by gaze pitch and gaze target - is mapped into Unity coordinate operations as follows. The four cases are visualized in Figure 5.7. In all four cases the lateral delta Δx is routed to the world X axis, rotated to align with the user's camera-right vector so that "right of the pen" always means "right of the user's head". The forward delta Δz is routed differently per case:

- **Wall Mapping (Gaze Straight + Looking at Object):** The system applies $\text{move} = \text{camRight} \cdot \Delta x + \text{Vector3.up} \cdot \Delta z$. Pushing the pen forward across the desk lifts the object up in the world. This matches the user's mental model when looking at a vertical surface like a picture frame.
- **Floor Mapping (Gaze Down + Looking at Object):** The system applies $\text{move} = \text{camRight} \cdot \Delta x + \text{camForward} \cdot \Delta z$. Pushing the pen forward pushes the object away across the floor. This matches the model of looking down at furniture on a floor.
- **Minimap Mapping (Looking at Pen):** The system uses one of two minimap-aligned mappings depending on the locked gaze pitch.
 - *Gaze Straight:* top-down XZ minimap (Δz maps to world Z).
 - *Gaze Down:* vertical-plane minimap, toggleable between front-facing XY ($\Delta z \rightarrow Y$) and side-facing YZ ($\Delta x \rightarrow Z$, $\Delta z \rightarrow Y$).

The wrist twist of the pen - extracted as `Mathf.DeltaAngle(startPenRotationY, currentY)` and amplified by `indirectRotationMultiplier  $\approx 3$` - is routed exclusively to the object's Y rotation. This decouples horizontal rotation from translation: the user can spin a chair while sliding it without ever tipping it forward. Two implementation details deserve emphasis, both addressing failure modes we observed during development. First, the gaze pitch is locked at the moment a drag begins (`lockedPosture = currentPosture` inside `StartDrag`). Once locked, the user can move their head freely during the drag without the routing changing. An earlier version updated posture continuously and flickered every time the user nodded slightly; the per-drag lock is what makes the routing feel deliberate rather than noisy. Second, the gaze-target component is updated continuously, but every switch triggers `ReAnchor()` (§5.3.2) to prevent visual teleportation.

5.5 Proxies, Visual Feedback, and Tools

Why proxies

Direct surface input requires the user's pen to be in contact with the part of the world they want to manipulate. In an interior-design scene, the things to be manipulated are 3D furniture items floating in a room, none of which the user can physically touch from a seated position. The system therefore needs a way to bring distant content into reach without changing where the user is sitting. Three alternatives exist: walk to the object (rejected because it conflicts with seated use), instantiate a held proxy, or render a lightweight stand-in inside the user's existing workspace. We chose the third, and instantiate two complementary proxies: a 2D shadow on the desk for every interactable, and a transient on-pen minimap for depth refinement. Both proxies are gaze-mediated: the shadow proxy is permanent but its targetability is gaze-driven (looking at the shadow lights it up and lets the pen engage with it), while the on-pen minimap is the one that is gaze-triggered and transient (it appears when the user looks at the pen and disappears when gaze shifts away). Beyond the two proxies, the application also relies on a layer of lightweight visual cues that make the underlying state machine legible to the user.

Why this set of visual feedback subsystems

The FSM and the axis routing make the system's state machine-readable; they do not make it human-readable. We therefore added a small number of visual cues, each in response to a specific failure we observed during development. The axis arrows make the routing concrete: without them, the first time the user enters the wall mapping they push the pen the wrong way. The controller floor shadow gives an analog depth cue against an otherwise textureless desk surface; without it, users miss the desk by 2–3 cm and lose engagement. The table grid provides scale and motion cues during long indirect drags; without it, fast pen movements feel weightless. None of these subsystems is individually novel, but each one is load-bearing.

Subsystem details

The Shadow Proxy: Generated at scene start by `ShadowManager`. A `ShadowFollower` snaps its position to $(target.x, globalTableHeight, target.z)$ and scales to the ob-

ject's XZ bounding box. Using a trigger collider allows a `Physics.SphereCastAll` from the gaze to hit the shadow even when the desk surface is between the user and the geometry (Figure 5.8).



Figure 5.8: Shadow proxies in the scene. Every interactable object casts a flat 2D handle onto the desk surface; gazing or manipulating an object lights its shadow with the same outline color.

The On-Pen Minimap Proxy: A Quad attached to the right controller displaying a `RenderTexture`. The `RenderTexture` is captured by a secondary camera with a lower field of view (FOV) than the player's main camera, resulting in a zoomed-in perspective on the quad (Figure 5.9).

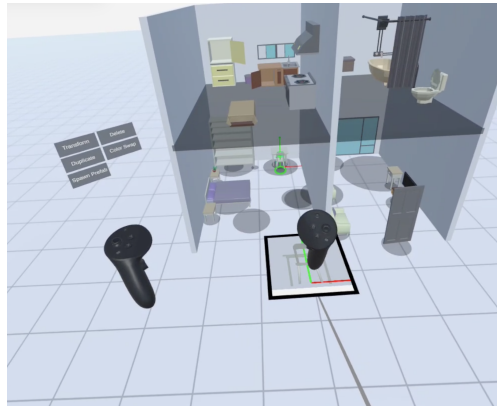


Figure 5.9: The on-pen minimap proxy during indirect manipulation. The wrist-anchored quad displays a backward-forward orthographic view of the active object, activated when the user looks at the pen.

The Outline System: Uses two colors: yellow for hovering and green for manipulation. Outline parameters are re-applied every frame during state changes to prevent flicker.

The Axis Arrows: `AxisFeedbackManager` generates X (red), Y (green), and Z (blue) arrows using `_ZTest Always`. During a drag, only the arrows representing *currently free* axes are displayed.

- Depth-and-Grounding Cues:**
- **ControllerShadowGrid:** a soft-edged circle under the pen that scales with altitude.
 - **TableGridGenerator:** a 128×128 grid 1 mm above the desk to provide scale and motion cues.
 - **GazeRayVisualizer:** draws the ray from 10 cm below the headset to avoid obstructing central vision.
 - **ControllerOverlay:** forces the controller’s render queue to 5000 to prevent clipping into the physical desk.

The Tool Model: Managed by `InteractionToolManager` with five tools: *Move*, *Delete*, *Duplicate*, *ColorPicker*, and *Spawn*. Tools are changed via a wrist-anchored hand menu toggled by the *X* button on the left controller. The tool affects only what the trigger does on a focused target; movement semantics are unchanged across tools, which keeps the FSM small.

5.6 Implementation Notes and Limitations

The system runs at a stable frame rate on the Quest Pro by bounding per-frame physics work (one `OverlapSphere` and one `SphereCastAll` per frame). Calibration is handled by `TableAutoCalibrator`, allowing the user to set desk height by pressing the pen against the real desk while holding the *A* button. Known limitations include:

1. No application-level user evaluation. The controlled study (Chapter 4) isolates each of the four atomic techniques, but the composed workflow with the FSM, the gaze-driven axis routing, the shadow and minimap proxies, has not been formally evaluated with users. Subjective claims about implicit mode switching feeling natural are based on informal observation only.
2. Shadow proxy overlap in dense scenes. When two interactable objects share an XZ footprint (e.g. a lamp on top of a side table), their shadow proxies overlap on the desk. The targeting tie-breaker resolves only the simultaneous Interactable - Shadow collision, not Shadow - Shadow collisions, so dense layouts can become harder to address from the desk.
3. Flat, horizontal desk assumption. `TableAutoCalibrator` registers a single horizontal plane height for the desk surface; the system has no support for sloped, curved, or multi-level surfaces, which restricts deployment to standard desks.
4. No bimanual manipulation. The non-dominant hand is reserved for the wrist-anchored tool menu (`HandMenuController`); the system does not support bimanual gestures such as two-handed scaling or rotation, which are common affordances in hand-tracked XR baselines.
5. No undo/redo for destructive actions. Deletions and color changes are committed immediately. A miscued Delete gesture requires the user to manually re-spawn and re-position the object.
6. No state persistence. The room layout resets between sessions; the application has no save/load workflow, which limits its use for long-form design tasks.

7. The system is currently limited to right-handed use for object manipulation, though it is designed for easy remapping to the left controller with minimal code modifications.
8. Hardware dependence. The system relies on the specific combination of integrated eye tracking and a pressure-sensitive stylus tip on a hand-held controller. Few current consumer XR headsets ship with both, which constrains the deployment surface beyond the Meta Quest Pro used here.

5.7 Reflection and Lessons Learned

Building the application surfaced a set of lessons that were not visible from the design space alone. We collect them here because they shaped the final system in ways the deeper technical sections cannot capture.

Small thresholds determine whether the system feels usable

The two parameters that took the longest to tune were the sticky-gaze buffer duration (settled at 300ms) and the hysteresis band on the gaze ray's vertical angle (17° entering Straight, 27° entering Down). Neither value appears in the abstract design space, and neither is structurally important. But cutting the gaze buffer to zero makes the system feel twitchy at distance, and collapsing the hysteresis band to a single threshold makes the axis routing flicker under natural micro-jitter of the gaze ray. Both parameters were added in response to specific failures observed during development, and both are the difference between a system that feels deliberate and one that feels noisy. The general lesson: implicit-input systems live or die on these temporal smoothing constants, and they deserve to be treated as primary design parameters rather than as last-mile polish.

The two indirect modes split coarse and fine adjustment

The application supports four coexisting interaction primitives, but they are not used uniformly across the workflow. The two indirect modes carry most of the work, and they trade off cleanly between speed and precision. IndirectAir - the trigger pressed while gaze is on a distant Interactable, with the pen held in mid-air - is the fastest mode: it requires no contact with the desk, engages immediately on trigger press, and the visual-angle gain of §5.3.1 turns small mid-air motions into large object motions. It is the right tool for coarse placement, where the user wants to drop an object roughly where it belongs and refine later. For fine adjustment, however, IndirectTableObject - the same gaze on the same distant object, but with the pen pressed onto the desk - is the precision sweet spot. The combination of stable hand support, the pressure-based engagement gesture that the desk physically absorbs (§5.5), and the gaze-and-posture axis routing of §5.4 together carry the weight of detailed alignment work. The two modes complement each other in a coarse-then-fine pattern that the application is built around: IndirectAir for the rough first pass, IndirectTableObject for the alignment. The Direct+3DOF and Direct+2DOF modes remain valuable for the nearby cases they were designed for but are used less often in a room-scale workflow operated from a single seated position.

The architectural refactor was forced by the visual subsystems, not by the manipulation logic

The original HybridPenController class held everything in one file. It became unmaintainable not because the manipulation logic was complex - the FSM remained small - but because the visual-feedback subsystems (outline updates, axis arrows, controller shadow, gaze ray) were interleaved with the physics queries frame by frame. Any change to a visual cue risked interrupting the targeting logic. The current three-handler split isolates physics-heavy code from cosmetic updates and is what makes the system extensible. The general lesson is that, in a real-time interaction system, the visual-feedback layer is the part most likely to need rapid iteration, and the architecture should isolate it from the parts that must remain stable.

What we would do differently

Three changes would land first if we rebuilt the application. First, an application-level user evaluation would happen in parallel with implementation rather than being deferred to future work; the absence of one is the most significant limitation of the current chapter, and many of the design decisions reported here would benefit from empirical support beyond informal observation. Second, the system would be symmetric for left- and right-handed users from the start, rather than left for later code-modification work as the current build requires. Third, the depth-perception problem would be attacked directly—with auto-snap-to-nearby-surface behavior or stereoscopic parallax cues—rather than indirectly with the layered visual cues; the cues help but do not fully solve the problem.

5.8 Outlook

Two open application-level problems

Depth perception on flat, textureless surfaces remains the most error-prone part of the workflow even with the floor shadow, the gridded desk, and the on-pen minimap. The indirect depth cues help but do not collectively reach the robustness of binocular parallax in real-world reach; stereoscopic auto-snap-to-nearby-surface behavior is the obvious next direction. The on-pen minimap is contested: some informal users adopt it as a natural extension of the desk metaphor, while others find that looking at the pen mid-drag breaks their focus on the object. We do not currently know which side is right, and the question deserves a controlled comparison before the design is generalized.

From application-level to thesis-level implications

The design lessons it surfaces - which signals to route, which transitions to leave implicit, when to support input with a surface, and how to proxy distant content - are consolidated in §6.6, where each lesson is tied to either a measured study result, an application observation, or both.

Toward formal application evaluation

The single most important next step is the application-level user study that §5.6 flags as the chapter's largest limitation. The composition, the implicit routing, the layered proxies, and the temporal smoothing constants all rest on informal observation rather than empirical measurement. A study that holds the controlled-comparison task constant and varies whether the techniques are exposed in isolation or as the composed FSM would be the most direct test of the design's central claim.

Chapter 6

Discussion and Future Work

6.1 Overview of Findings

Our study evaluated four multimodal VR interaction techniques: GazeMidAir, GazeTable, StylusMidAir, and StylusTable, across performance, physical demand, and user experience metrics. The results confirm that all four techniques are viable for 2D panel interaction in VR, yet each presents a distinct profile of strengths and trade-offs. Addressing RQ1, gaze-assisted selection improved acquisition speed and positional accuracy in MidAir interaction, though it introduced synchronization challenges in the Table condition. Addressing RQ2, surface-supported stylus input yielded greater manipulation precision and physical comfort compared to MidAir input, at the cost of increased hand movement and slower task completion. Addressing RQ3, the four techniques occupy distinct positions in a multidimensional space of speed, precision, physical effort, and visual effort, with no single technique dominating across all dimensions. Notably, no significant differences were found across NASA-TLX workload measures, suggesting that techniques differ in how effort is distributed rather than in total workload imposed.

6.2 RQ1: Effect of Gaze-Assisted Selection

Accuracy Benefits of Gaze Pre-selection

RQ1 asked how gaze-assisted selection affects acquisition speed, task completion time, and selection errors in stylus-based XR manipulation. The results present a nuanced picture: gaze assistance improved accuracy in MidAir interaction but introduced a meaningful synchronization cost in the Table condition.

Acquisition Speed and Task Completion Time

Gaze-assisted techniques showed a trend toward being slower than their Stylus counterparts in acquisition speed, though neither difference reached statistical significance. GazeMidAir (0.79 s) was marginally slower than StylusMidAir (0.77 s), and GazeTable (0.92 s) was slower than StylusTable (0.85 s). This suggests that gaze pre-selection does not provide a speed benefit over direct stylus pointing, and in the Table condition

shows a trend toward incurring additional overhead, though this did not reach statistical significance. The speed cost in GazeTable is likely attributable to the late-trigger synchronization effect: because gaze naturally anticipates upcoming targets, it tends to drift away from the currently fixated object before the confirming surface tap is performed, causing missed selections that require recovery time. P3 and P9 described this as "unconsciously shifting toward the target before pressing, causing misses," and P3 further noted that "GazeTable requires more effort to go back with gaze after a miss than just adjusting a cursor." This error-recovery cost, rather than any inherent slowness of gaze-based selection, likely explains the speed disadvantage of GazeTable relative to StylusTable.

Selection Errors and Positional Accuracy

Gaze-assisted selection provided a clear accuracy benefit in MidAir interaction. Gaze MidAir achieved significantly lower positional error (1.83 cm) than StylusMidAir (2.39 cm, $p \leq .001$), suggesting that offloading target acquisition to gaze reduces the precision demands placed on hand movement. When the hand is no longer responsible for pointing at a target, it can focus exclusively on smooth object displacement, resulting in more accurate final placement. This decoupling of acquisition and manipulation represents a meaningful functional advantage for precision-sensitive tasks. In the Table condition, however, gaze assistance did not confer the same accuracy benefit. GazeTable (1.50 cm) achieved lower positional error than GazeMidAir but was comparable to StylusTable (1.47 cm), and the Technique $\times$ Size interaction between GazeTable and StylusTable ($p = .027$) indicates that GazeTable's accuracy relative to StylusTable degrades for smaller targets - consistent with participants' reports of difficulty selecting small targets with gaze (P2, P3, P9). Together, these findings suggest that the accuracy benefit of gaze assistance is most reliable in MidAir conditions, where it is not undermined by the late-trigger synchronization effect that characterizes surface-based confirmation.

6.3 RQ2: Effect of Surface-Supported Stylus Input

RQ2 asked how surface-supported stylus input affects manipulation precision, hand movement, and physical comfort compared with mid-air stylus input. The results show a consistent pattern: Table input offers greater precision and perceived comfort, while MidAir input is faster and requires less hand displacement, but is perceived as more physically demanding.

6.3.1 Manipulation Precision

Table variants achieved lower positional error than MidAir variants. StylusTable (1.47 cm) was the most accurate technique overall, and participants attributed this to its absolute spatial mapping, which makes input predictable and repeatable. P2 and P9 regarded StylusTable as the most accurate technique, with P2 attributing this to its absolute spatial mapping, which made interactions easy to repeat and predict. The physical stability provided by the table surface grounds the hand during manipulation, reducing unintended movement and enabling finer motor control than is possible when the hand is unsupported in mid-air. StylusMidAir (2.39 cm), by contrast, was described

by participants as the least precise technique due to the high sensitivity of its ray-casting input (P2, P7), which amplifies small wrist tremors into large cursor displacements on the panel.

Hand Movement

Despite the precision advantage of Table interaction, it incurred significantly greater hand movement (GazeTable: 0.24 m; StylusTable: 0.24 m) compared to MidAir techniques (GazeMidAir: 0.17 m; StylusMidAir: 0.09 m), with StylusMidAir exhibiting the least movement of all techniques (all $p \leq .008$). This situation that, greater movement in the more stable condition, is best explained by the constrained input space of Table interaction. Unlike MidAir interaction, which can extend freely in space, the table surface is a bounded horizontal area; when the stylus reaches the edge, the user must lift and reposition it, generating movement that does not contribute to output displacement. The interaction effect between Technique and Distance further indicates that StylusMidAir is less affected by longer target distances than the other techniques (all $p \leq .001$), and GazeMidAir is less affected than GazeTable ($p = .028$), suggesting that the surface boundary becomes a more significant constraint as target distance increases.

Physical Comfort

Despite requiring more hand movement, Table conditions were consistently preferred in terms of physical comfort. Participants appreciated the physical support offered by the surface, with P11 noting the ability to "rest my arm while doing it." MidAir conditions, by contrast, were frequently flagged as fatiguing (P2, P4, P6, P7, P10, P11), with P3 specifically identifying StylusMidAir as "especially tiring for the wrist." This underscores a critical limitation of using hand movement distance as a proxy for physical demand: the nature of motion matters as much as its magnitude. Sustained wrist rotation in mid-air, without any surface for support, loads distal joints continuously in a way that aggregate displacement metrics do not reflect. The physical grounding of the table surface thus provides a comfort benefit that persists even when it comes at the cost of greater total hand movement.

6.4 RQ3: Trade-offs Between Speed, Precision, Physical Effort, and Visual Effort

RQ3 asked what trade-offs emerge between speed, precision, physical effort, and eye fatigue across gaze-assisted and stylus-only interaction in mid-air and table-based input spaces. The results reveal a multidimensional trade-off space in which no single technique dominates, and where objective performance and subjective experience diverge in instructive ways.

6.4.1 Speed and Precision

The most consistent trade-off observed across techniques is between speed and precision. MidAir techniques were faster (all $p \leq .022$) but less accurate (all $p \leq .02$) than their Table counterparts. Within MidAir conditions, GazeMidAir achieved higher

accuracy than StylusMidAir ($p < .001$) at a marginal speed cost, representing a partial resolution of this trade-off: gaze pre-selection improves precision without substantially sacrificing speed. Within Table conditions, GazeTable and StylusTable were comparable in accuracy, with StylusTable performing slightly better for small targets due to the late-trigger effect in GazeTable. Overall, the speed-precision trade-off is most acute when comparing across surface conditions, and is partially mitigated by gaze assistance within the MidAir condition.

6.4.2 Physical Effort and Speed

MidAir techniques required less hand movement and rotation yet were perceived as more physically fatiguing than Table techniques, particularly due to the absence of physical support for the arm and wrist. This dissociation between movement quantity and perceived effort highlights that physical demand in VR interaction is not simply a function of displacement: joint loading, posture, and the availability of external support all contribute to perceived fatigue in ways that movement metrics do not fully capture. The speed advantage of MidAir interaction thus comes with a physical cost that is not reflected in kinematic data alone.

6.4.3 Physical Effort and Visual Effort

A significant difference in perceived eye fatigue was found across techniques ($\chi^2(12) = 25.436$, $p < .001$, $W = .983$), with Stylus variants perceived as less eye-fatiguing than Gaze variants (all $p \leq .023$). Crucially, no corresponding differences were found across NASA-TLX workload dimensions. This pattern supports an effort-redistribution account of multimodal interaction: engaging gaze for target selection shifts effort from the physical (hand and arm) to the visual system, without increasing total workload. Gaze techniques thus offer a mechanism for relieving physical demand at the cost of increased visual engagement a trade-off that may be favourable in contexts where arm fatigue is a primary concern, but less so in visually demanding environments or for users prone to eye strain.

6.4.4 Subjective Preference and Objective Performance

Perhaps the most practically significant trade-off is between objective efficiency and subjective preference. While no statistically significant differences were found in post-study preference ratings, qualitative trends were consistent: StylusMidAir received the lowest mean preference score (3.67) despite being among the fastest techniques, while StylusTable received the highest (5.08) despite being the slowest. This divergence suggests that users do not simply prefer the most efficient technique. Instead, preference appears to be driven by perceived comfort, predictability, and a sense of physical control - qualities that Table interaction provides through haptic grounding and absolute spatial mapping, and that MidAir interaction lacks. For designers, this implies that optimising for speed alone risks producing systems that are fast but uncomfortable, and that user adoption may depend as much on felt stability as on measured efficiency.

6.5 Summary

StylusMidAir is fast and requires minimal hand movement, but is the least accurate and most wrist-fatiguing technique. Best suited to quick, low-precision tasks. **GazeMidAir** offers competitive speed with improved accuracy over StylusMidAir, and redistributes effort from the hand to the visual system. Best suited to tasks requiring both speed and accuracy in mid-air. **StylusTable** is the most accurate and highest-preferred technique, with predictable absolute mapping, but is the slowest and incurs the most hand movement. Best suited to precision tasks and extended sessions. **GazeTable** offers comparable precision to StylusTable but is undermined by the late-trigger synchronization effect, particularly for small targets. Its potential advantages - reduced reach distance and upper-arm fatigue - are currently offset by this usability challenge.

6.6 Design Implications

The findings of this thesis arise from three converging sources - the controlled four-technique comparison of Chapter 4, the hybrid application of Chapter 5, and the conceptual framework of Chapter 3. This section consolidates the design lessons that emerge across them, written for a reader designing the next gaze-stylus-surface system rather than as a description of our own. Each implication is grounded in at least one of: a measured study result, an observation from application development, or a positioning relative to prior work.

11. Compose techniques rather than choose between them

The most consistent message of Chapter 4 is the absence of a dominant technique. StylusMidAir was fastest in manipulation time and required the least hand movement and rotation, but produced the highest positional error. GazeMidAir delivered the fastest acquisition and significantly lower positional error than StylusMidAir ($p < .001$), but added eye fatigue ($p \leq .023$). StylusTable yielded the lowest positional error and the highest comfort ratings, but the slowest acquisition. GazeTable matched StylusTable on precision but degraded on small targets through a Technique $\times$ Size interaction ($p = .027$). NASA-TLX workload did not differ between any pair, and preference did not yield a statistical winner. Each technique occupies a distinct position in the speed-precision-effort space, and the design question is not which to pick but how to combine them. The application of Chapter 5 demonstrates that the design space can be instantiated as a composed workflow on commodity hardware, with the application's manipulation modes transitioning between each other without an explicit mode UI. Current commercial XR systems — Apple Vision Pro's Gaze+Pinch, Meta Quest 3's hand-tracked or controller input, the Logitech MX Ink stylus — commit to a single dominant paradigm and forgo the complementarity that combination provides; research systems such as SymbiosisSketch [15] and VRSketchIn [29] get closer but require a second held device. Mode coexistence, not mode replacement, is the design goal.

I2. Use gaze for selection when acquisition speed or positional accuracy matters in the air

RQ1 is the cleanest gaze-related finding in the thesis. GazeMidAir achieved both faster acquisition and significantly lower positional error than StylusMidAir ($p < .001$), with no measured workload cost on NASA-TLX. The benefit is consistent with the broader gaze-based-selection literature [7, 8]: offloading targeting to gaze relieves the hand of pointing responsibility, which lets the hand specialize for stable manipulation. The benefit is conditional, however. Gaze raised eye fatigue across all conditions ($p \leq .023$), and on a surface it gained no precision over the stylus while introducing a small-target failure mode (the GazeTable Technique $\times$ Size interaction). The actionable rule is therefore narrower than “use gaze for selection”: use gaze for selection in the air, when fast acquisition or low positional error are the targets; do not assume the benefit transfers to surface conditions or to small-target work.

I3. Use surface support when stable precision matters more than speed

RQ2 produced a consistent surface-precision trade-off. Both Table techniques produced significantly lower positional error than both MidAir techniques ($p \leq .02$ for the relevant comparisons) and received the highest subjective comfort ratings, but both were also slower across timing measures ($p \leq .022$). Cheng et al. [4] report the same surface-precision benefit across a range of tabletop and mid-air UIs, and our results extend that finding to a controlled XR docking task. The implication is that surface support is the right primitive when the workflow phase prioritizes stable, repeatable, sub-centimeter placement; mid-air input is the right primitive when the phase prioritizes quick coarse reach. The two strengths are complementary rather than competing.

I4. Drive mode routing from continuously available implicit signals

A system that wants to combine selection methods (I1) and switch between input spaces (I3) needs a routing mechanism, and the design choice is what that mechanism reads. The application uses signals already produced by the user during the task - gaze (both its target and the vertical angle of the gaze ray), pen-tip proximity to virtual objects, and pen-tip pressure on the desk - and demands no additional input action for mode transitions. The empirical justification is the same RQ1 result that motivates I2: gaze is already a measurable, accurate target-selection signal in mid-air, so reusing it as the routing signal for axis assignment (§5.4) and proxy activation (§5.5) costs nothing extra. The pattern is consistent with Gaze+Pinch [8] (gaze for selection without an explicit gesture) and Gaze-shifting [21] (gaze as the direct-indirect switch), generalized to a multi-axis routing problem rather than a single binary switch. The practical caveat is that implicit signals are usable only when smoothed in time. The 300 ms sticky-gaze buffer and the $17^\circ/27^\circ$ hysteresis band on the gaze ray’s vertical angle that the application required to feel stable are not last-mile polish but primary design parameters; any system adopting implicit routing should budget for similar temporal infrastructure.

15. Sequence the workflow from coarse mid-air to fine surface input

I1, I2, and I3 together imply a temporal structure for the workflow, not just a static set of techniques to expose. The MidAir-fast / Table-accurate split (RQ2) is most useful when the two are sequenced: a fast mid-air placement pass followed by a precise surface refinement pass. The application encodes this as `INDIRECTAIR` for the rough first pass and `INDIRECTTABLEOBJECT` for alignment, with implicit transition between them driven by whether the pen is held in the air or pressed onto the desk. This mirrors the established desktop pattern of free placement followed by snap-to-grid refinement, but applies it across input spaces rather than within a single one. Commercial XR design tools currently sit at one extreme of this split - Gravity Sketch and similar mid-air sketchers, or iPad-CAD tools such as Shapr3D - and a unified system that supports both phases is a strict superset of either.

16. Use adaptive, visual-angle gain when working distances vary

The Chapter 4 study used a fixed 1:5 control-display gain for both Table conditions because the task operated at a single, well-defined distance. Outside a controlled experiment, however, the distance from the user to the manipulated object varies continuously — in our application, from arm’s reach to the far wall of a 2 m room from a single seated position. The application generalizes the fixed gain to a continuously evaluated visual-angle gain (eye-to-object distance divided by eye-to-hand distance), which keeps the angular motion of the object proportional to the angular motion of the hand regardless of where each is. This is the version of Casiez et al.’s [23] adaptive-gain finding that requires no calibration. The implication is broader than our application: any indirect XR input that has to span a non-trivial range of distances should default to an adaptive gain, and reserve a fixed gain only for the special case of a single well-defined working distance.

17. Mix permanent ambient proxies with transient gaze-triggered proxies for distant content

Distant content in a room-scale XR scene cannot be reached directly from a seated position, and the design question is what kind of proxy to use. The application uses two that solve complementary problems. The shadow proxy is permanent and ambient: one per interactable, spawned at scene start, always visible on the desk as a flat darkened patch beneath each floating object. It is cheap to render, looks like a naturalistic shadow rather than UI, and turns the desk into an addressable surface for any object in the room with no management overhead. The on-pen minimap proxy is the opposite: transient and foreground, activated only when gaze settles on the pen during indirect manipulation, and dismissed when gaze leaves. It carries a higher visual cost while active but imposes none when not. Together they trace two distinct points in the proxy design space, both of which avoid the cognitive cost of permanent-and-foreground proxies (e.g., World-in-Miniature overviews [27] that occupy space whether the user needs them or not) and the gestural cost of explicit-instantiation proxies (e.g., VRSketchIn portals [29], which the user must draw before use). We recommend choosing along the same axis when designing proxies for any XR workflow: permanent-but-ambient for proxies that should be available everywhere at low attentional cost, and transient-but-foreground for

proxies that serve a specialized perspective the user invokes deliberately. A system that uses both is strictly more flexible than a system that uses only one.

6.7 Discussion on The Application

The controlled study in this thesis characterizes four interaction techniques in isolation, establishing their individual performance across speed, precision, physical effort, and visual effort. A natural question that follows is whether these profiles retain their meaning when the techniques are no longer isolated - that is, when a user moves between them within a single continuous task. To explore this question, Chapter 5 presents a hybrid VR interior-design application in which the two design dimensions the four atomic techniques span - selection method and input space - are instantiated as a continuous 3D workflow, with implicit transitions driven by pen-tip pressure, gaze target, and gaze-ray angle rather than by explicit mode switches.

Building the composed system surfaced design problems that the controlled study could not reveal. Most notably, implicit-input systems of this kind depend on temporal smoothing parameters - in particular, the sticky-gaze buffer duration and the hysteresis band on gaze-ray angle - that have no analogue in an isolated-technique evaluation. These parameters determine whether the system feels deliberate or noisy, yet they are invisible to the metrics of the controlled study. This points to a limitation of atomic technique evaluation more broadly: performance measurements obtained in isolation do not predict the transition costs and smoothing requirements that emerge when the same techniques are composed. The transition cost between techniques - how much time and effort is lost when a user implicitly switches from IndirectAir to IndirectTableObject mid-task - is an empirical question that the present study does not address, and that a future evaluation of the composed system would be best placed to answer.

Taken together, the application chapter demonstrates that the four atomic techniques characterized in this study can be composed on commodity XR hardware, and that their empirically established trade-off profiles translate into a coherent design rationale for when each technique should be active. At the same time, it exposes the limits of controlled-study findings as a basis for application design: temporal parameters, transition costs, and the ergonomic dynamics of switching between techniques all require evaluation in the composed context. The most important next step is therefore a user study of the application itself - one that holds the task constant and varies whether the techniques are experienced in isolation or as the composed finite state machine - to determine whether composition preserves, amplifies, or undermines the individual advantages that the controlled study identified.

6.8 Limitations

Several limitations should be considered when interpreting these findings. The study was conducted in a controlled environment using a fixed selection-and-placement task; results may not generalize to more complex or varied VR interaction scenarios. With 12 participants, the study was sufficiently powered to detect large effects in a within-subjects design but limits broader generalization across user populations. The study used a standard VR controller held as a stylus, and the ergonomic discomfort

reported by multiple participants suggests that results may differ with purpose-designed hardware. No longitudinal component was included, leaving the effects of learning, fatigue accumulation, and muscle memory development over extended use unexplored. The late-trigger effect in GazeTable was not systematically mitigated in the current design; a version with improved synchronization mechanisms may yield meaningfully different results for that technique. Finally, the study did not measure physiological correlates of fatigue (e.g., electromyography, eye-tracking metrics), relying instead on subjective ratings, which may underestimate differences in physical and visual load.

6.9 Future Work

Several directions for future research emerge from this work. The synchronization challenge in GazeTable warrants dedicated investigation; future work should evaluate improved confirmation mechanisms, such as dwell-time thresholds, predictive gaze filtering, or multimodal confirmation signals, and assess their impact on accuracy, error recovery, and eye fatigue. Longitudinal studies examining performance and fatigue over extended use sessions would complement the present single-session findings, particularly with respect to the development of muscle memory for Table-based techniques and the long-term effects of wrist loading in MidAir conditions. Evaluation of purpose-designed VR stylus hardware is warranted given the consistent ergonomic concerns raised by participants, as hardware design may substantially alter the comfort and preference profiles observed here. Future work could also explore adaptive or hybrid strategies that dynamically switch between techniques based on task demands - for example, defaulting to GazeMidAir for rapid selection and StylusTable for high-precision placement, potentially combining the advantages of multiple modalities within a single interface. Finally, a within-subject comparison between the four techniques as the study exposes them and the same four techniques as the application composes them would test whether the implicit FSM, the gaze-driven routing, and the layered proxies deliver workflow-level benefits beyond what each technique offers in isolation. A positive result would convert the design implications in §6.6 from informed proposals into measured guidelines; a null result would identify which of the application’s design choices carry their weight and which do not.

Chapter 7

Conclusion

This thesis investigated four multimodal VR interaction techniques: GazeMidAir, GazeTable, StylusMidAir, and StylusTable, for 2D object manipulation on a virtual panel. A controlled within-subjects user study with 12 participants, each completing 256 trials per condition across varying target sizes, distances, and diagonal orientations, yielded data across seven dependent measures analyzed using Aligned Rank Transform repeated-measures ANOVA. Three research questions were addressed. RQ1 established that gaze-assisted selection improves positional accuracy in MidAir interaction by decoupling acquisition from manipulation, but introduces synchronization challenges in the Table condition that offset this benefit. RQ2 established that surface-supported stylus input provides greater manipulation precision and physical comfort than MidAir input, at the cost of slower task completion and increased hand movement due to input space constraints. RQ3 characterized the multidimensional trade-off space across techniques, revealing that gaze assistance redistributes effort from the physical to the visual system without increasing total workload, and that subjective preference is driven by comfort and predictability rather than objective efficiency.

Beyond the controlled study, the thesis also presents a hybrid VR interior-design application that situates the four atomic techniques within a continuous seated workflow. The application demonstrates that the four techniques characterized in isolation can be reassembled on commodity XR hardware using signals the user already produces during the task, such as gaze (target and ray angle), pen-tip proximity to virtual objects, and pen-tip pressure on the desk. The same design surfaces two further mechanisms: a gaze-mediated axis routing that resolves the dimensional gap between a 2D pen and a 3D scene, and a layered proxy strategy that pairs a permanent shadow handle on the desk with a transient on-pen minimap for distant content.

Taken together, the controlled comparison and the application argue against the framing of XR interaction as a search for a single dominant technique. Each of the four techniques occupies a distinct position in the speed-precision-effort space, and the more productive question is how to compose them. This thesis offers measured evidence for the trade-offs that govern that composition, a system that demonstrates the composition is buildable on commodity hardware, and a set of design implications that translate both into actionable guidance for future gaze, stylus, and surface-supported XR systems.

Bibliography

- [1] Mar Gonzalez-Franco and Andrea Colaco. Guidelines for productivity in virtual reality. *Interactions*, 31(3):46–53, 2024.
- [2] Esen K. Tütüncü, Mar Gonzalez-Franco, and Eric J. Gonzalez. Handover: Enabling precise selection & manipulation of 3d objects with mouse and hand tracking. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*, UIST '25. Association for Computing Machinery, 2025.
- [3] Chen Chen, Matin Yarmand, Zhuoqun Xu, Varun Singh, Yang Zhang, and Nadir Weibel. Investigating input modality and task geometry on precision-first 3d drawing in virtual reality. In *2022 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pages 384–393. IEEE, 2022.
- [4] Yi Fei Cheng, Tiffany Luong, Andreas Rene Fender, Paul Streli, and Christian Holz. Comfortable user interfaces: Surfaces reduce input error, time, and exertion for tabletop and mid-air user interfaces. In *2022 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pages 150–159. IEEE, 2022.
- [5] Jason L. Reisman, Philip L. Davidson, and Jefferson Y. Han. A screen-space formulation for 2d and 3d direct manipulation. In *Proceedings of the 22nd Annual ACM Symposium on User Interface Software and Technology*, UIST '09, pages 69–78. Association for Computing Machinery, 2009.
- [6] Han Shi, Hanzhong Luo, HyeonBeom Yi, and Seungwoo Je. Reachpad: Interacting with multiple virtual screens using a single physical pad through haptic retargeting. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, CHI '25. Association for Computing Machinery, 2025.
- [7] Alexander Plopski, Teresa Hirzle, Nahal Norouzi, Long Qian, Gerd Bruder, and Tobias Langlotz. The eye in extended reality: A survey on gaze interaction and eye tracking in head-worn extended reality. *ACM Computing Surveys*, 55(3):1–39, 2022.
- [8] Ken Pfeuffer, Benedikt Mayer, Diako Mardanbegi, and Hans Gellersen. Gaze + pinch interaction in virtual reality. In *Proceedings of the 5th Symposium on Spatial User Interaction*, SUI '17, pages 99–108. Association for Computing Machinery, 2017.
- [9] Baosheng James Hou, Yasmeen Abdrabou, Florian Weidner, and Hans Gellersen. Unveiling variations: A comparative study of vr headsets regarding eye tracking

- volume, gaze accuracy, and precision. In *2024 IEEE Conference on Virtual Reality and 3D User Interfaces Abstracts and Workshops (VRW)*, pages 650–655. IEEE, 2024.
- [10] Yeji Park, Jiwan Kim, and Ian Oakley. Gazehandsync: Mitigating late-trigger errors for seamless gaze-hand interactions. In *Proceedings of the 2025 Symposium on Eye Tracking Research and Applications, ETRA '25*. Association for Computing Machinery, 2025.
 - [11] Uta Wagner, Jinwook Kim, Zhikun Wu, Qiushi Zhou, Mario Romero, Alessandro Iop, Tiare Feuchtner, and Ken Pfeuffer. A study of multimodal pen + gaze interaction techniques for shape point translation in extended reality. In *2025 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pages 454–464. IEEE, 2025.
 - [12] Ravin Balakrishnan and Gordon Kurtenbach. Exploring bimanual camera control and object manipulation in 3d graphics interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 56–62. Association for Computing Machinery, 1999.
 - [13] Uta Wagner, Andreas Asferg Jacobsen, Tiare Feuchtner, Hans Gellersen, and Ken Pfeuffer. Eye-hand movement of objects in near space extended reality. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology, UIST '24*. Association for Computing Machinery, 2024.
 - [14] Wenxin Sun, Mengjie Huang, Chenxin Wu, Rui Yang, Ji Han, and Yong Yue. Evaluating the feeling of control in virtual object translation on 2d interfaces. *AI EDAM*, 37:e9, 2023.
 - [15] Rahul Arora, Rubaiat Habib Kazi, Tovi Grossman, George Fitzmaurice, and Karan Singh. Symbiosissketch: Combining 2d & 3d sketching for designing detailed 3d objects in situ. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, pages 1–15, 2018.
 - [16] Ken Pfeuffer, Abdullatif Dinc, Jan Obernolte, Radiah Rivu, Yasmeen Abdrabou, Franziska Shelter, Yomna Abdelrahman, and Florian Alt. Bi-3d: Bi-manual pen-and-touch interaction for 3d manipulation on tablets. In *The 34th Annual ACM Symposium on User Interface Software and Technology*, pages 149–161, 2021.
 - [17] Hongyu Mao, Mar Gonzalez-Franco, Vrushank Phadnis, Eric J. Gonzalez, and Ishan Chatterjee. Restfulraycast: Exploring ergonomic rigging and joint amplification for precise hand ray selection in xr. In *Proceedings of the Designing Interactive Systems Conference*, pages 28–39. Association for Computing Machinery, 2025.
 - [18] Clifton Forlines and Ravin Balakrishnan. Evaluating tactile feedback and direct vs. indirect stylus input in pointing and crossing selection tasks. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, CHI '08*, pages 1563–1572. Association for Computing Machinery, 2008.

- [19] Eugenie Brasier, Olivier Chapuis, Nicolas Ferey, Jeanne Vezien, and Caroline Appert. ARPads: Mid-air indirect input for augmented reality. In *2020 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pages 332–343. IEEE, 2020.
- [20] Uta Wagner, Mathias N. Lystbæk, Pavel Manakhov, Jens Emil Sloth Grønbæk, Ken Pfeuffer, and Hans Gellersen. A fitts’ law study of gaze-hand alignment for selection in 3d user interfaces. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI ’23. Association for Computing Machinery, 2023.
- [21] Ken Pfeuffer, Jason Alexander, Ming Ki Chong, Yanxia Zhang, and Hans Gellersen. Gaze-shifting: Direct-indirect input with pen and touch modulated by gaze. In *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology*, UIST ’15, pages 373–383. Association for Computing Machinery, 2015.
- [22] Yang Liu, Qiushi Zhou, Mathias N Lystbæk, Aidan Kehoe, Mario Gutierrez, Hans Gellersen, and Ken Pfeuffer. Stylusport: Investigating teleportation using stylus in vr. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, pages 1–12, 2026.
- [23] Géry Casiez, Daniel Vogel, Ravin Balakrishnan, and Andy Cockburn. The impact of control-display gain on user performance in pointing tasks. *Human–computer interaction*, 23(3):215–250, 2008.
- [24] Xiaolong Liu, Lili Wang, Shuai Luan, Xuehuai Shi, and Xinda Liu. Distant object manipulation with adaptive gains in virtual reality. In *2022 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pages 739–747. IEEE, 2022.
- [25] Fengyuan Zhu, Ludwig Sidenmark, Mauricio Sousa, and Tovi Grossman. Pinch-lens: Applying spatial magnification and adaptive control-display gain for precise selection in virtual reality. In *2023 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pages 1221–1230. IEEE, 2023.
- [26] Kenneth P Herndon, Robert C Zeleznik, Daniel C Robbins, D Brookshire Conner, Scott S Snibbe, and Andries Van Dam. Interactive shadows. In *Proceedings of the 5th annual ACM symposium on User interface software and technology*, pages 1–6, 1992.
- [27] Richard Stoakley, Matthew J Conway, and Randy Pausch. Virtual reality on a wim: interactive worlds in miniature. In *Proceedings of the SIGCHI conference on Human factors in computing systems*, pages 265–272, 1995.
- [28] Jin-Du Wang, Ke Zhou, Haoyu Ren, Per Ola Kristensson, and Xiang Li. Handows: A palm-based interactive multi-window management system in virtual reality. *IEEE Transactions on Visualization and Computer Graphics*, 2025.

- [29] Tobias Drey, Jan Gugenheimer, Julian Karlbauer, Maximilian Milo, and Enrico Rukzio. Vrsketchin: Exploring the design space of pen and tablet interaction for 3d sketching in virtual reality. In *Proceedings of the 2020 CHI conference on human factors in computing systems*, pages 1–14, 2020.
- [30] Shintaro Imatani, Kensuke Tobitani, and Kyo Akabane. Touchwim: Object manipulation in ar spatial design with world in miniature and hybrid user interface. *IEEE Access*, 2025.
- [31] Ryo Suzuki, Hooman Hedayati, Clement Zheng, James L Bohn, Daniel Szafir, Ellen Yi-Luen Do, Mark D Gross, and Daniel Leithinger. Roomshift: Room-scale dynamic haptics for vr with furniture-moving swarm robots. In *Proceedings of the 2020 CHI conference on human factors in computing systems*, pages 1–11, 2020.
- [32] Rumeysa Turkmen, Zeynep Ecem Gelmez, Anil Ufuk Batmaz, Wolfgang Stuerzlinger, Paul Asente, Mine Sarac, Ken Pfeuffer, and Mayra Donaji Barrera Machuca. Eyeguide & eyeconguide: Gaze-based visual guides to improve 3d sketching systems. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, pages 1–14, 2024.
- [33] Uta Wagner, Matthias Albrecht, Andreas Asferg Jacobsen, Haopeng Wang, Hans Gellersen, and Ken Pfeuffer. Gaze, wall, and racket: Combining gaze and hand-controlled plane for 3d selection in virtual reality. *Proceedings of the ACM on Human-Computer Interaction*, 8(ISS):189–213, 2024.
- [34] Miao Wang, Zi-Ming Ye, Jin-Chuan Shi, and Yang-Liang Yang. Scene-context-aware indoor object selection and movement in vr. In *2021 IEEE Virtual Reality and 3D User Interfaces (VR)*, pages 235–244. IEEE, 2021.
- [35] Samantha Aziz, Dillon J. Lohr, Lee Friedman, and Oleg Komogortsev. Evaluation of eye tracking signal quality for virtual reality applications: A case study in the meta quest pro. In *Proceedings of the 2024 Symposium on Eye Tracking Research and Applications*, ETRA '24. Association for Computing Machinery, 2024.
- [36] Sandra G. Hart. NASA-task load index (NASA-TLX); 20 years later. In *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, volume 50, pages 904–908, 2006.
- [37] Samuel S. Shapiro and Martin B. Wilk. An analysis of variance test for normality. *Biometrika*, 52(3/4):591–611, 1965.
- [38] Jacob O. Wobbrock, Leah Findlater, Darren Gergle, and James J. Higgins. The aligned rank transform for nonparametric factorial analyses using only ANOVA procedures. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '11, pages 143–146. Association for Computing Machinery, 2011.
- [39] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

- [40] Milton Friedman. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association*, 32(200):675–701, 1937.
- [41] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.
- [42] Dennis Wolf, Jan Gugenheimer, Marco Combosch, and Enrico Rukzio. Understanding the heisenberg effect of spatial interaction: A selection induced error for spatially tracked input devices. In *Proceedings of the 2020 CHI conference on human factors in computing systems*, pages 1–10, 2020.
- [43] Géry Casiez, Nicolas Roussel, and Daniel Vogel. 1 euro filter: a simple speed-based low-pass filter for noisy input in interactive systems. In *Proceedings of the SIGCHI Conference on human factors in computing systems*, pages 2527–2530, 2012.

Appendix A

Technical Details of the Application

This appendix consolidates implementation-level reference material for the hybrid VR interior-design application of Chapter 5. It is intended as a compact lookup for a reader who has already read the chapter and wants to find a state transition, a routing rule, a hardware version, or a specific numerical constant without re-reading prose. Nothing in this appendix introduces new design rationale; it tabulates and lists what Chapter 5 describes inline.

A.1 The Interaction State Machine

Table A.1 enumerates the six states of the application’s finite state machine, the gating conditions that drive entry into each state, the tag of the focused target required for that state to engage, and the handler call that owns the per-frame behaviour while the state is active. The system is in exactly one state per frame. Each active state is held while its gating input is held; releasing the gating input returns the system to IDLE. There is no direct transition between active states.

State	Gating signals	Focus tag	Handler call
Idle	<i>none</i>	<i>none</i>	DeterminePotentialTarget
DirectAir	Trigger and pen $\cap$ object	Interactable	HandleDirectAir
DirectTableShadow	Pressure and pen $\cap$ shadow	Shadow	HandleDirectTableShadow
IndirectAir	Trigger and gaze on object	Interactable	HandleIndirectAir
IndirectTableShadow	Pressure and gaze on shadow	Shadow	HandleIndirectTableShadow
IndirectTableObject	Pressure and gaze on object	Interactable	HandleIndirectTableObject

Table A.1: FSM state reference. *Trigger* = controller trigger pressed; *Pressure* = pen-tip force on the desk above threshold; $\cap$ = collider overlap between the pen tip and the target.

Table A.2 lists the classes that implement the FSM and the supporting subsystems described in §5.5. *HybridPenController* owns the state variable and the per-frame Update loop; it dispatches to three handler classes that own targeting, movement, and start/end-of-drag bookkeeping respectively. The remaining classes are supporting components used by the controller and the handlers.

Class	Role
HybridPenController	FSM state variable, per-frame Update, handler dispatch
PenTargetingHandler	Per-frame focused-target resolution from pen tip and gaze
PenMovementHandler	Per-state movement deltas and axis routing
PenInteractionHandler	Start/end-of-drag bookkeeping (grab offsets, locked posture)
GazeProvider	Smoothed gaze ray and 300 ms history buffer
ShadowManager	Spawns one ShadowFollower per Interactable at scene start
ShadowFollower	Snaps to (object.x, table.y, object.z) every frame
InteractionToolManager	Singleton; current tool (Move / Delete / Duplicate / Color / Spawn)
TableAutoCalibrator	Sets desk-plane height from a calibration press
AxesFeedbackManager	Renders X / Y / Z axis arrows on the grabbed object
HandMenuController	Wrist-anchored tool menu
ControllerShadowGrid	Soft-edged altitude indicator under the pen
TableGridGenerator	128×128 grid above the desk for scale and motion cues
GazeRayVisualizer	Renders the gaze ray from 10 cm below the headset
ControllerOverlay	Forces the controller’s render queue to 5000 to prevent desk clipping

Table A.2: Class decomposition of the manipulation subsystem.

A.2 Gaze-Dependent Axis Routing

The axis-routing rules of §5.4 are determined by two two-valued signals: the gaze ray’s vertical angle (Straight or Down, classified through the hysteresis band of §A.3) and the current gaze target (the manipulated object, or the pen itself). The lateral input delta Δx always maps to the world X axis, rotated by the camera-right vector so that “right of the pen” tracks “right of the head”. The forward input delta Δz is routed differently per case (Table A.3).

Gaze pitch	Gaze target	Mapping	Output direction
Straight	Object	Wall	$\Delta z \rightarrow$ world Y (up)
Down	Object	Floor	$\Delta z \rightarrow$ world Z (away)
Straight	Pen	Top-down minimap	$\Delta z \rightarrow$ world Z
Down	Pen	Vertical-plane minimap, front-facing	$\Delta z \rightarrow$ world Y
Down	Pen	Vertical-plane minimap, side-facing	$\Delta x \rightarrow$ world Z , $\Delta z \rightarrow$ world Y

Table A.3: Forward-delta routing as a function of gaze pitch and gaze target. The vertical-plane minimap (last two rows) is togglable between front- and side-facing views; the front view is the default.

The wrist twist of the pen, extracted as `Mathf.DeltaAngle(startPenRotationY, currentY)` and amplified by `indirectRotationMultiplier  $\approx 3$` , is routed exclusively to the object’s Y rotation. Two further mechanisms make the routing usable in practice. First, the gaze-derived posture is locked at the moment a drag begins, so that natural eye and head movement during manipulation does not change the mapping mid-motion. Second, when the gaze target flips between Object and Pen mid-drag, `ReAnchor()` re-bases the pen-from-anchor delta against the current pen and object positions, so that the object does not visually teleport at the moment the routing switches.

A.3 Implicit-Signal Smoothing and Constants

Five temporal-smoothing and threshold constants determine whether the implicit signals feel deliberate or noisy (Table A.4). Each one was added in response to a specific failure mode observed during development.

Constant	Value	Failure prevented
Sticky-gaze buffer duration	300 ms	Acquired target lost to ocular micro-jitter
Posture hysteresis (enter Straight)	$\theta < 17^\circ$	Routing flicker at the threshold
Posture hysteresis (enter Down)	$\theta > 27^\circ$	Routing flicker at the threshold
Pen-tip pressure threshold	≈ 0.15 on <code>Axis1D.PrimaryStylusForce</code>	Spurious desk-contact engagements
Table indirect gain multiplier	1.30	Under-reach during desk-pressing modes

Table A.4: Temporal-smoothing and threshold constants used by the manipulation subsystem.

The visual-angle gain of Chapter 5 is evaluated every frame as the ratio of two scalar distances:

```
gain = (eye - object).magnitude / (eye - pen).magnitude;
if (state.IsTablePressing()) gain *= tableIndirectGainMultiplier;
appliedDelta = gain * rawPenDelta;
```

The sticky-gaze buffer is implemented as a rolling 300 ms history of focused-target identifiers. The currently acquired target is the most recent target whose continuous focus duration crosses the buffer threshold; if no target meets that bar, the most recently acquired target is held for one additional grace interval before being released:

```
dwellMs = 0
foreach sample in history.LastMilliseconds(300):
    if sample.target == candidate: dwellMs += sample.dt
if dwellMs >= 300:                acquired = candidate
else if (now - lastAcquireTime) < graceMs:    acquired = lastAcquired
else:                                         acquired = none
```

The posture classification is a one-dimensional hysteresis on the gaze ray's vertical angle θ :

```
if posture == Straight and theta > 27 degrees: posture = Down
else if posture == Down and theta < 17 degrees: posture = Straight
```

The asymmetric thresholds create a 10° dead band between the two states; neither threshold is crossed by natural gaze drift, which is what prevents flicker when the user is looking near the boundary.

A.4 Hardware and Software Stack

The application targets the Meta Quest Pro running the Meta XR All-in-One SDK on Unity 6. Table A.5 lists the input, tracking, and physics primitives the implementation depends on. No external trackers and no second tracked device are used.

Component	Identifier
Headset	Meta Quest Pro
Controller	Meta Touch Pro (right hand, inverted stylus grip)
Game engine	Unity 6
XR SDK	Meta XR All-in-One SDK
Pen-tip pressure axis	<code>OVRInput.Axis1D.PrimaryStylusForce</code>
Trigger axis	<code>OVRInput.Axis1D.PrimaryIndexTrigger</code>
Eye-tracking API	<code>OVREyeGaze</code> (per-eye gaze ray)
Gaze smoothing	One Euro filter [?]
Physics queries per frame	one <code>Physics.OverlapSphere</code> (pen tip), one <code>Physics.SphereCastAll</code> (gaze)

Table A.5: Hardware and software components used by the application.

The desk is an ordinary physical desk and is registered into Unity coordinates by `TableAutoCalibrator`: the user holds the A button on the right controller and presses the pen tip against the real desk surface, and the calibrator records the resulting global Y value as the desk-plane height. The calibration is persistent for the session and survives scene reloads.

A.5 Reproduction

The full source code, the Unity project, and the prefab library used in the application are available at <https://github.com/platak1sm/MScThesis>. A demonstration video of the implemented workflow is available at <https://youtu.be/bcVJdNiHlLo>. To reproduce on physical hardware, clone the repository, open the project in Unity 6 (or later) with the Meta XR All-in-One SDK installed, switch the build target to Android, and deploy to a Meta Quest Pro with eye tracking enabled in the headset’s privacy settings. The first run on a new desk requires a calibration press as described in §A.4; no other setup is required.